*Last updated on **Apr 7, 2026***

Lakeflow Jobs

Lakeflow Jobs is workflow automation for Databricks, providing orchestration for data processing workloads so that you can coordinate and run multiple tasks as part of a larger workflow. You can optimize and schedule the execution of frequent, repeatable tasks and manage complex workflows.

This article introduces concepts and choices related to managing production workloads using Lakeflow Jobs.

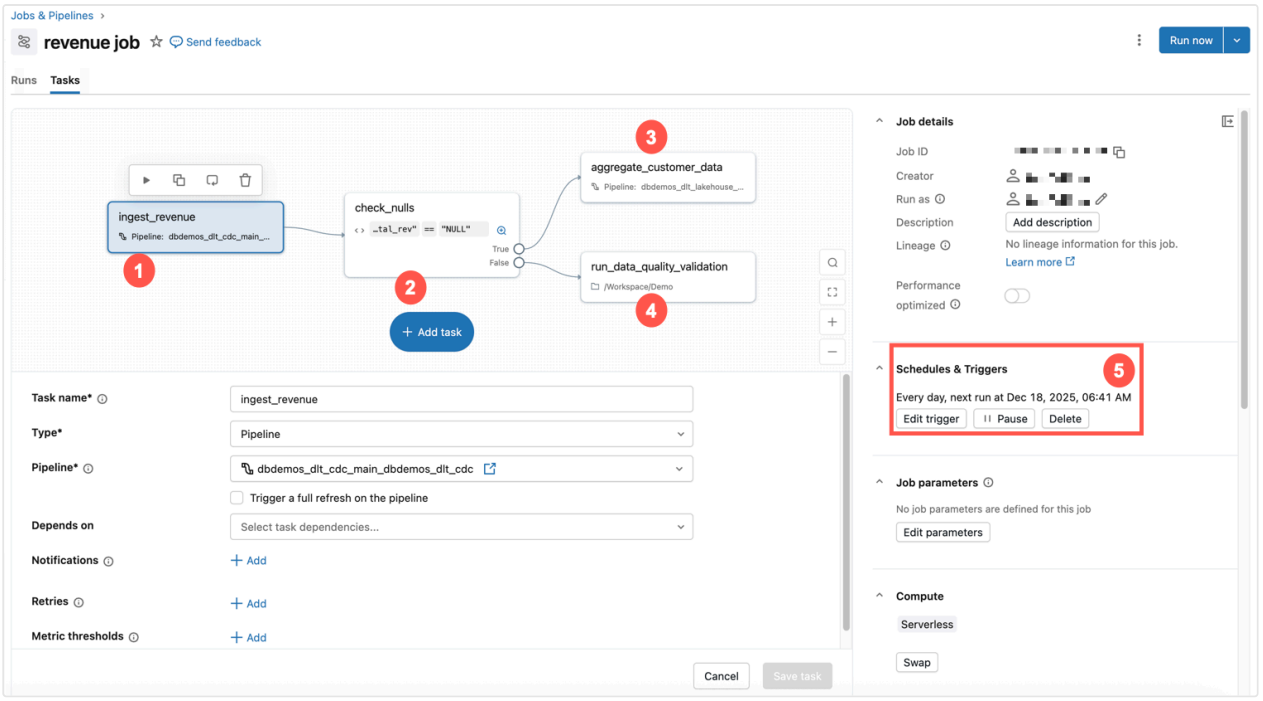
What are jobs?

In Databricks, a job is used to schedule and orchestrate tasks on Databricks in a workflow. Common data processing workflows include ETL workflows, running notebooks, and machine learning (ML) workflows, as well as integrating with external systems like dbt.

Jobs consist of one or more tasks, and support custom control flow logic like branching (if / else statements) or looping (for each statements) using a visual authoring UI. Tasks can load or transform data in an ETL workflow, or build, train and deploy ML models in a controlled and repeatable way as part of your machine learning pipelines.

Example: Daily data processing and validation job

The example below shows a job in Databricks.



This example job has the following characteristics:

1. The first task ingests revenue data.
2. The second task is an if / else check for nulls.
3. If not, then a transformation task is run.
4. Otherwise, it runs a notebook task with a data quality validation.
5. It is scheduled to run at the same time every day.

To get a quick introduction to creating your own job, see Create your first workflow with Lakeflow Jobs.

Common use cases

From foundational data engineering principles to advanced machine learning and seamless tool integration, these common use cases showcase the breadth of capabilities that drive modern analytics, workflow automation, and infrastructure scalability.

Orchestration concepts

There are three main concepts when using Lakeflow Jobs for orchestration in Databricks: jobs, tasks, and triggers.

Job - A job is the primary resource for coordinating, scheduling, and running your operations. Jobs can vary in complexity from a single task running a Databricks notebook to hundreds of tasks with conditional logic and dependencies. The tasks in a job are visually represented by a Directed Acyclic Graph (DAG). You can specify properties for the job, including:

- Trigger - this defines when to run the job.
- Parameters - run-time parameters that are automatically pushed to tasks within the job.
- Notifications - emails or webhooks to be sent when a job fails or takes too long.
- Git - source control settings for the job tasks.

Task - A task is a specific unit of work within a job. Each task can perform a variety of operations, including:

- A notebook task runs a Databricks notebook. You specify the path to the notebook and any parameters that it requires.
- A pipeline task runs a pipeline. You can specify existing Lakeflow Spark Declarative Pipelines, such as a materialized view or streaming table.
- A Python script tasks runs a Python file. You provide the path to the file and any necessary parameters.

There are many types of tasks. For a complete list, see Types of tasks. Tasks can have dependencies on other tasks, and conditionally run other tasks, allowing you to create complex workflows with conditional logic and dependencies.

Trigger - A trigger is a mechanism that initiates running a job based on specific conditions or events. A trigger can be time-based, such as running a job at a scheduled time (for example, ever day at 2 AM), or event-based, such as running a job when new data arrives in cloud storage.

Monitoring and observability

Jobs provide built-in support for monitoring and observability. The following topics give an overview of this support. For more details about monitoring jobs and orchestration, see Monitoring and observability for Lakeflow Jobs.

Job monitoring and observability in the UI – In the Databricks UI you can view jobs, including details such as the job owner and the result of the last run, and filter by job properties. You can view a history of job runs, and get detailed information about each task in the job.

Job run status and metrics – Databricks reports job run success, and logs and metrics for each task within a job run to diagnose issues and understand performance.

Notifications and alerts – You can set up notifications for job events via email, Slack, custom webhooks and a host of other options.

Custom queries through system tables – Databricks provides system tables that record job runs and tasks across the account. You can use these tables to query and analyze job performance and costs. You can create dashboards to visualize job metrics and trends, to help monitor the health and performance of your workflows.

Limitations

The following limitations exist:

- A workspace is limited to 2000 concurrent task runs. A `429 Too Many Requests` response is returned when you request a run that cannot start immediately.
- The number of jobs a workspace can create in an hour is limited to 10000 (includes "runs submit"). This limit also affects jobs created by the REST API and notebook workflows.
- A workspace can contain up to 12000 saved jobs.
- A job can contain up to 1000 tasks.
- When tasks use dynamic values in their parameters, job parameters are limited to 10,000 characters.

Can I manage workflows programmatically?

Databricks has tools and APIs that allow you to schedule and orchestrate your workflows programmatically, including the following:

- Databricks CLI
- Declarative Automation Bundles
- Databricks extension for Visual Studio Code
- Databricks SDKs
- Jobs REST API

For examples of using tools and APIs to create and manage jobs, see Automate job creation and management. For documentation on all available developer tools, see Local development tools.

External tools use the Databricks tools and APIs to programmatically schedule workflows. For example, you can also schedule your jobs using tools such as Apache Airflow. See Orchestrate Lakeflow Jobs with Apache Airflow.

Last updated on ***Apr 16, 2026***

Configure and edit Lakeflow Jobs

You can create and run a job using the Jobs UI, or developer tools such as the Databricks CLI or the REST API. Using the UI or API, you can repair and rerun a failed or canceled job. This article shows how to create, configure, and edit jobs using the **Jobs & Pipelines** workspace UI. For information about other tools, see the following:

- To learn about using the Databricks CLI to create and run jobs, see Databricks CLI.
- To learn about using the Jobs API to create and run jobs, see Jobs in the REST API reference.
- If you prefer an infrastructure-as-code (IaC) approach to configuring jobs, you can use Declarative Automation Bundles. To learn about using bundles to configure and orchestrate your jobs, see Declarative Automation Bundles.
- To learn how to run and schedule jobs directly in a Databricks notebook, see Create and manage scheduled notebook jobs.

> **TIP**
>
> To view a job as YAML, click the kebab menu to the left of **Run now** for the job and then click **Switch to code version (YAML)**.

What is the minimum configuration needed for a job?

All jobs on Databricks require the following:

- A task that contains logic to be run, such as a Databricks notebook. See Configure and edit tasks in Lakeflow Jobs

- A compute resource to run the logic. The compute resource can be serverless compute, classic jobs compute, or all-purpose compute. See Configure compute for jobs.
- A specified schedule for when the job should be run. Optionally, you can omit setting a schedule and trigger the job manually.
- A unique name.

Create a new job

This section describes the steps to create a new job with a notebook task and schedule with the workspace UI.

Jobs contain one or more tasks. You create a new job by configuring the first task for that job.

> **NOTE**
>
> Each task type has dynamic configuration options in the workspace UI. See Configure and edit tasks in Lakeflow Jobs.

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Click **Create**, then **Job**.
3. Click the **Notebook** tile to configure the first task. If the **Notebook** tile is not available, click **Add another task type** and search for **Notebook**.
4. Enter a **Task name**.
5. Select a notebook for the **Path** field.
6. Click **Create task**.

If your workspace is not enabled for serverless compute for jobs, you must select a **Compute** option. Databricks recommends always using jobs compute when configuring tasks.

A new job appears in the workspace jobs list with the default name `New Job <date> <time>`.

You can continue to add more tasks within the same job, if needed for your workflow. Jobs with greater than 100 tasks may have special requirements. For more information,

see Jobs with a large number of tasks.

Scheduling a job

You can decide when your job is run. By default, it will only run when you manually start it, but you can also configure it to run automatically. You can create a trigger to run a job on a schedule, or based on an event.

Controlling the flow of tasks within the job

When configuring multiple tasks in jobs, you can use specialized tasks to control how the tasks run. See Control the flow of tasks within Lakeflow Jobs.

Select a job to edit in the workspace

To edit an existing job with the workspace UI, do the following:

1. In your Databricks workspace's sidebar, click **Jobs & Pipelines**.
2. Optionally, select the **Jobs** and **Owned by me** filters.
3. Click your job's **Name** link.

Use the jobs UI to do the following:

- Edit job settings
- Rename, clone, or delete a job
- Add new tasks to an existing job
- Edit task settings

> **NOTE**
>
> You can also view the JSON definitions for use with REST API get, create, and reset endpoints.

Edit job settings

The side panel contains the **Job details**. You can change the job schedule or trigger, job parameters, compute configuration, tags, notifications, the maximum number of concurrent runs, duration thresholds, and Git settings. You can also edit job permissions if job access control is enabled.

Add parameters for all job tasks

Parameters configured at the job level are passed to the job's tasks that accept key-value parameters, including Python wheel files configured to accept keyword arguments. See Parameterize jobs.

Add tags to a job

To add labels or key-value attributes to your job, you can add *tags* when you edit the job. You can use tags to filter jobs in the Jobs list. For example, you can use a `department` tag to filter all jobs that belong to a specific department.

> **NOTE**
>
> Because job tags are not designed to store sensitive information such as personally identifiable information or passwords, Databricks recommends using tags for non-sensitive values only.

Tags also propagate to job clusters created when a job is run, allowing you to use tags with your existing cluster monitoring.

Click **+ Tag** in the **Job details** side panel to add or edit tags. You can add the tag as a label or key-value pair. To add a label, enter the label in the **Key** field and leave the **Value** field empty.

Use Git with jobs

You can configure job tasks to check out source code directly from a remote Git repository. For instructions and best practices, including sparse checkout for large repositories, see Use Git with Lakeflow Jobs.

Add a serverless usage policy to a job

> **PREVIEW**
>
> This feature is in Public Preview.

If your workspace uses serverless usage policies to attribute serverless usage, you can select your jobs's serverless usage policy using the **Budget policy** setting in the **Job details** side panel. See Attribute usage with serverless usage policies.

Rename, clone, or delete a job

To rename a job, go to the jobs UI and click the job name.

You can quickly create a new job by cloning an existing job. Cloning a job creates an identical copy of the job except for the job ID. To clone a job, do the following:

1. Click Jobs & Pipelines on the left sidebar.
2. Click the name of the job you want to clone to open the Jobs UI.
3. Click ⋮ next to the **Run now** button.
4. Select **Clone job** from the drop-down menu.
5. Enter a name for the cloned job.
6. Click **Clone**.

Delete a job

To delete a job, go to the job page, click ⋮ next to the job name, and select **Delete job** from the drop-down menu.

Configure thresholds for job run duration or streaming backlog metrics

> **PREVIEW**
>
> Streaming observability for Lakeflow Jobs is in Public Preview.

You can configure optional thresholds for job run duration or streaming backlog metrics. To configure duration or streaming metric thresholds, click **Duration and streaming backlog thresholds** in the **Job details** panel.

To configure job duration thresholds, including expected and maximum completion times for the job, select **Run duration** in the **Metric** drop-down menu. Enter a duration in the **Warning** field to configure the job's expected completion time. If the job exceeds this threshold, an event is triggered. You can use this event to notify when a job is running slowly. See Configure notifications for slow jobs. To configure a maximum completion time for a job, enter the maximum duration in the **Timeout** field. If the job does not complete in this time, Databricks sets its status to "Timed Out".

To configure a threshold for a streaming backlog metric, select the metric in the **Metric** drop-down menu and enter a value for the threshold. To learn about the specific metrics supported by a streaming source, see View metrics for streaming tasks.

If an event is triggered because a threshold is exceeded, you can use the event to send a notification. See Configure notifications for slow jobs.

You can optionally specify duration thresholds for tasks. See Configure thresholds for task run duration or streaming backlog metrics.

Enable queueing of job runs

> **NOTE**
>
> Queueing is enabled by default for jobs created through the UI after April 15, 2024.

To prevent runs of a job from being skipped because of concurrency limits, you can enable queueing for the job. When queueing is enabled, the run is queued for up to 48 hours if resources are unavailable for a job run. When capacity is available, the job run is dequeued and run. Queued runs are displayed in the runs list for the job and the recent job runs list.

A run is queued when one of the following limits is reached:

- The maximum concurrent active runs in the workspace.
- The maximum concurrent `Run Job` task runs in the workspace.
- The maximum concurrent runs of the job.

Queueing is a job-level property that queues runs only for that job.

To enable or disable queueing, click **Advanced settings** and click the **Queue** toggle button in the **Job details** side panel.

Configure maximum concurrent runs

By default, the maximum concurrent runs for all new jobs is 1.

Click **Edit concurrent runs** under **Advanced settings** to set this job's maximum number of parallel runs.

Databricks skips the run if the job has already reached its maximum number of active runs when attempting to start a new run.

Set this value higher than 1 to allow multiple concurrent runs of the same job. This is useful, for example, if you trigger your job on a frequent schedule and want to enable consecutive runs to overlap or trigger multiple runs that differ by their input parameters.

*Last updated on **Apr 27, 2026***

Automating jobs with schedules and triggers

In Lakeflow Jobs, it is possible to configure jobs to automatically trigger in any of the following situations:

- On a time-based schedule
- On the arrival of files to a Unity Catalog storage location
- Continuously

You can also trigger job runs manually or through external orchestration tools.

Job schedules and triggers

Trigger type	Behavior
Scheduled	Triggers a job run based on a time-based schedule. See Run jobs on a schedule.
Table update	Triggers a job run when source tables are updated. See Trigger jobs when source tables are updated.
File arrival	Triggers a job run when new files arrive in a monitored Unity Catalog storage location. See Trigger jobs when new files arrive.
Continuous	To keep the job always running, trigger another job run whenever a job run completes or fails. See Run jobs continuously.
None (manual)	Runs are triggered manually with the **Run now** button or programmatically using other orchestration tools. See

Trigger type	Behavior
	single job run

By default, only a single run of a job can be active at a time. However, it is possible to increase this limit in the Advanced settings. Runs are skipped when they exceed the configured max concurrency for a job. See Configure maximum concurrent runs.

Configure a trigger on a job

1. Open the job that you want to configure a trigger on.
2. In the **Job details** pane, scroll down to the **Schedules & Triggers** section, and then click **Add trigger**.
3. In **Schedules & Triggers**, select the type of trigger you want to configure: **Scheduled**, **Table update**, **File arrival**, or **Continuous**.

   Based on the type of trigger, other options are available to configure, as well.
4. Click **Save**. After you save your trigger, your job starts only when a new file arrives in the configured location.

> **NOTE**
>
> If one or more tasks in a job with multiple tasks are unsuccessful, you can re-run the subset of unsuccessful tasks. See Re-run failed and skipped tasks.

Pause and resume job triggers

You can pause and resume your jobs in the **Job details** pane for your job under **Schedules & Triggers**. The **Pause** and **Resume** buttons appear only for jobs that have a trigger configured.

To pause any active job trigger, click **Pause**. When you pause a trigger, any currently active runs continue, but the trigger no longer starts new runs.

To resume the trigger, click **Resume**. When you resume a trigger, the configured behavior resumes on the same schedule previously configured.

When creating or editing a trigger, you can also control these settings in the **Schedules & Triggers** dialog. Toggle between the **Active** and **Paused** to control the **Trigger Status**.

> **NOTE**
>
> If a run is active when a continuous trigger is resumed, the job scheduler waits until that run is complete to trigger a new run.

Last updated on ***Apr 30, 2026***

Control the flow of tasks within Lakeflow Jobs

Some jobs are simply a list of tasks that need to be completed. You can control the execution order of tasks by specifying dependencies between them. You can configure tasks to run in sequence or parallel.

However, you can also create branching flows that include conditional tasks, error correction, or cleanup. Lakeflow Jobs provides functionality to control the flow of tasks within a job. The following topics describe ways that you can control the flow of your tasks.

Retries

Retries specify how many times a particular task should be re-run if the task fails with an error message. Errors are often transient and resolved through restart. Some features on Databricks, such as schema evolution with Structured Streaming, assume that you run jobs with retries to reset the environment and allow a workflow to proceed.

If you specify retries for a task, the task restarts up to the specified number of times if it encounters an error. Not all job configurations support task retries. See Set a retry policy.

When running in continuous trigger mode, Databricks automatically retries with exponential backoff. See How are failures handled for continuous jobs?.

Run if conditional tasks

You can use the **Run if** task type to specify conditionals for later tasks based on the outcome of other tasks. You add tasks to your job and specify upstream-dependent tasks. Based on the status of those tasks, you can configure one or more downstream tasks to run. Jobs support the following dependencies:

- All succeeded
- At least one succeeded
- None failed
- All done
- At least one failed
- All failed

See Configure task dependencies

If/else conditional tasks

You can use the **If/else** task type to specify conditionals based on some value. See Add branching logic to a job with the If/else task.

Lakeflow Jobs support `taskValues` that you define in your logic and allow you to return the results of some computation or state from a task to the jobs environment. You can define **If/else** conditions against `taskValues`, job parameters, or dynamic values.

Lakeflow Jobs supports the following operands for conditionals:

- `==`
- `!=`
- `>`
- `>=`
- `<`
- `<=`

See also:

- Use task values to pass information between tasks
- What is a dynamic value reference?

- Parameterize jobs

For each tasks

Use the `For each` task to run another task in a loop, passing a different set of parameters to each iteration of the task.

To add a `For each` task to a job, you must define a `For each` task and a *nested task*. The nested task is the task to run for each iteration of the `For each` task and is one of the standard Databricks task types. Multiple methods are supported for passing parameters to the nested task.

See Use a `For each` task to run another task in a loop.

Disabled tasks

Disable a task to skip it at run time without removing it from the job. The task keeps its configuration and run history, and Lakeflow Jobs evaluates downstream tasks against their `Run if` conditions to determine whether they also run.

See Disabled tasks in Lakeflow Jobs.

Last updated on ***Apr 17, 2026***

Configure compute for jobs

This article contains recommendations and resources for configuring compute for Lakeflow Jobs.

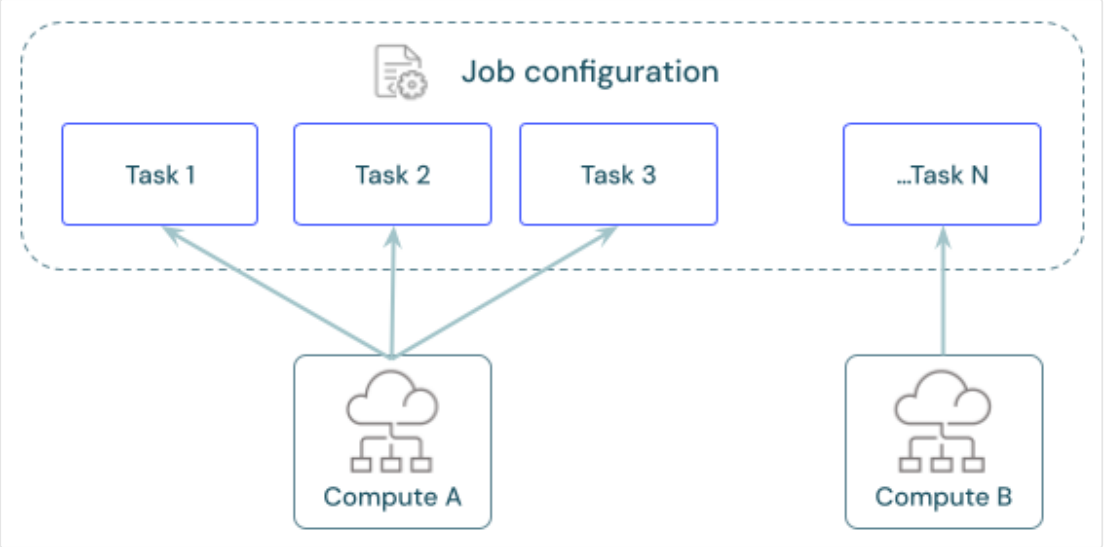
IMPORTANT

Limitations for serverless compute for jobs include the following:

- No support for **Continuous** scheduling.
- No support for default or time-based interval triggers in Structured Streaming.

For more limitations, see Serverless compute limitations.

Each job can have one or more tasks. You define compute resources for each task. Multiple tasks defined for the same job can use the same compute resource.



What is the recommended compute for each task?

The following table indicates the recommended and supported compute types for each task type.

> **NOTE**
>
> Serverless compute for jobs has limitations and does not support all workloads. See Serverless compute limitations.

Task	Recommended compute	Supported compute
Notebooks	Serverless jobs	Serverless jobs, classic jobs, classic all-purpose
Python script	Serverless jobs	Serverless jobs, classic jobs, classic all-purpose
Python wheel	Serverless jobs	Serverless jobs, classic jobs, classic all-purpose
SQL	Serverless SQL warehouse	Serverless SQL warehouse, pro SQL warehouse
Lakeflow Spark Declarative Pipelines	Serverless pipeline	Serverless pipeline, classic pipeline
dbt	Serverless SQL warehouse	Serverless SQL warehouse, pro SQL warehouse
dbt CLI commands	Serverless jobs	Serverless jobs, classic jobs, classic all-purpose
JAR	Classic jobs	Classic jobs, classic all-purpose
Spark Submit	Classic jobs	Classic jobs

Pricing for Lakeflow Jobs is tied to the compute used to run tasks. For more details, see Databricks pricing.

How do I configure compute for Jobs?

Classic jobs compute is configured directly from the Lakeflow Jobs UI, and these configurations are part of the job definition. All other available compute types store their configurations with other workspace assets. The following table has more details:

Compute type	Details
Classic jobs compute	You configure compute for classic jobs using the same UI and settings available for all-purpose compute. See Compute configuration reference.
Serverless compute for jobs	Serverless compute for jobs is the default for all tasks that support it. Databricks manages compute settings for serverless compute. See Run your Lakeflow Jobs with serverless compute for workflows.
SQL warehouses	Serverless and pro SQL warehouses are configured by workspace admins or users with unrestricted cluster creation privileges. You configure tasks to run against existing SQL warehouses. See Connect to a SQL warehouse.
Lakeflow Spark Declarative Pipelines compute	You configure compute settings for Lakeflow Spark Declarative Pipelines during pipeline configuration. See Configure classic compute for pipelines. Databricks manages compute resources for serverless Lakeflow Spark Declarative Pipelines. See Configure a serverless pipeline.
All-purpose compute	You can optionally configure tasks using classic all-purpose compute. Databricks does not recommend this configuration for production jobs. See Compute configuration reference and Should all-purpose compute ever be used for jobs?.

Share compute across tasks

Configure tasks to use the same jobs compute resources to optimize resource usage with jobs that orchestrate multiple tasks. Sharing compute across tasks can reduce latency associated with start-up times.

You can use a single job compute resource to run all tasks that are part of the job or multiple job resources optimized for specific workloads. Any job compute configured as part of a job is available for all other tasks in the job.

The following table highlights differences between job compute configured for a single task and job compute shared between tasks:

	Single task	**Shared across tasks**
Start	When the task run begins.	When the first task run configured to use the compute resource begins.
Terminate	After the task runs.	After the final task configured to use the compute resource runs.
Idle compute	Not applicable.	Compute remains on and idle while tasks not using the compute resource run.

A shared job cluster is scoped to a single job run and cannot be used by other jobs or runs of the same job.

Libraries cannot be declared in a shared job cluster configuration. You must add dependent libraries in task settings.

Shared driver state across tasks

When multiple tasks share a jobs compute resource, the tasks run on the same driver JVM. Class state and singletons persist across tasks for the duration of the job run. For most workloads this is transparent, but be aware of the following implications:

- **Scala singletons and companion objects are shared across tasks.** Mutable state in a Scala companion object persists between tasks that run on the same

shared cluster. If parallel tasks read from or write to the same companion-object variable, one task's value can overwrite another's. For a working example, see the Knowledge Base article Multi-task workflows using incorrect parameter values.

- **Libraries loaded by one task remain available to subsequent tasks** for the duration of the job run.

If your code requires task-level isolation, use one of the following approaches:

- Configure each task to use a separate jobs compute resource.
- Add explicit task dependencies so the tasks run sequentially rather than in parallel.
- Refactor the code to avoid relying on singleton or shared mutable state. For example, pass parameters explicitly to each function instead of reading them from a companion object.

Review, configure, and swap jobs compute

The **Compute** section in the **Job details** panel lists all compute configured for tasks in the current job.

Tasks configured to use a compute resource are highlighted in the task graph when you hover over the compute specification.

Use the **Swap** button to change the compute for all tasks associated with a compute resource.

Classic jobs compute resources have a **Configure** option. Other compute resources give you options to view and modify compute configuration details.

More information

For additional details on configuring Databricks classic jobs, see Best practices for configuring classic Lakeflow Jobs.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

Last updated on ***Apr 16, 2026***

Use Git with Lakeflow Jobs

Job tasks can check out source code directly from a remote Git repository.

The following task types support remote Git repositories:

- Notebooks
- Python scripts
- SQL files
- data build tool (dbt) projects

All tasks in a job must reference the same commit in the remote repository. When a job run begins, Databricks takes a snapshot of the specified branch or commit, so that all tasks in that run use the same version of the code.

When you view the run history of a task that runs code stored in a remote Git repository, the **Task run details** pane includes Git details, including the commit SHA associated with the run. See View task run history.

> **NOTE**
>
> Tasks configured to use a remote Git repository cannot write to workspace files. These tasks must write temporary data to ephemeral storage attached to the driver node and persistent data to a volume or table.

Using a Git repository source vs using Git folders.

This page discusses tasks that can pull source code directly from a remote Git repository. Workspaces also support a feature called Git folders, where a folder in your workspace is synced with a Git repository. A task can use a Git folder as

However, you must manage syncing with the repository. Using a remote Git repository as described here automatically pulls new source, if available, at job run time.

Databricks recommends referencing workspace paths in Git folders only for rapid iteration and testing during development. For staging and production jobs, configure tasks to reference a remote Git repository instead.

Configure a Git provider for a job

The jobs UI has a dialog to configure a remote Git repository. This dialog is accessible from the **Job details** pane under the **Git** heading, or in any task configured to use a **Git provider**. To access the dialog, click **Add Git settings** in the **Job details** pane.

In the **Git** dialog (labeled **Git information** if accessed during task configuration), enter the following details:

- The **Git repository URL**.
- Select your **Git provider** from the drop-down list.
- In the **Git reference** field, enter the identifier for a branch, tag, or commit that corresponds to the version of the source code you want to run.
- Select **branch**, **tag**, or **commit** from the drop-down.

You must specify only one of the following:

- **branch**: The name of the branch, for example, `main`.
- **tag**: The tag's name, for example, `release-1.0.0`.
- **commit**: The hash of a specific commit, for example, `e0056d01`.

> **NOTE**
>
> The dialog might prompt you with the following: **Git credentials for this account are missing. Add credentials**. You must configure a remote Git repository before using it as a reference. See Configure Git integration for Git folders.

When you view the run history of a task that runs code stored in a remote Git repository, the **Task run details** panel includes Git details, including the commit SHA associated with the run. See View task run history.

Sparse checkout for large repositories

For large repositories, you can use sparse checkout to import only specific directories rather than the full repository. Sparse checkout reduces checkout time and resource usage per job run.

However, improper configuration can cause cache fragmentation, which degrades execution times across your entire workspace. This section describes trade-offs and issues that might arise when using sparse checkout.

How Databricks caches repository checkouts

Databricks caches each Git checkout based on four values:

- Workspace
- Repository URL
- Exact commit hash
- Fingerprint of the sparse checkout pattern (the exact set of folder paths)

Any job run that matches all four criteria reuses a cache entry, which remains valid for up to one week. For example, if you have 3 different jobs, and they all have the same criteria, they use the same cache to the repository until there is a new commit (of after 1 week).

Every unique sparse checkout pattern creates a separate fingerprint, and therefore a separate cache entry. If 20 users each add a custom folder to their pattern, the system creates 20 distinct cache keys and imports the shared folder tree 20 times — multiplying load on your workspace. Creating a single sparse checkout pattern that includes all 20 of their folders (for example, is a parent folder), allows a single cache to work more often and have better performance in your jobs. The trade off is a larger number of files in your checkout.

Decide whether to use sparse checkout

Only enable sparse checkout if your use case meets both of the following criteria:

- **Size**: Your repository is large (for example, it exceeds 2,500 files).

- **Stable targeting**: The target branch is updated infrequently (for example, about one commit per hour or less). Avoid branches that change rapidly due to automated CI/CD workflows.

If you use sparse checkout, your organization should also adopt one or both of the following pattern strategies:

- **Standardization**: Use three or fewer shared checkout patterns across the organization to maximize cache hits.
- **Micro-targeting**: Structure patterns so that each targets a small number of files. For best performance, target fewer than 200 files.

These can help minimize your import rate.

Calculate your import rate

Before enabling sparse checkout, estimate your projected **Files Per Hour** import rate. Limits apply at the workspace level across all jobs and users.

Files Per Hour = Job Runs Per Hour × Cache Miss Rate × Files Imported Per Miss

Factor	What drives it
Job Runs Per Hour	Trigger frequency across all users
Cache Miss Rate	Commit frequency on the target branch and number of unique sparse patterns
Files Imported Per Miss	Total repository size or sparse checkout subset size

Example: 180 runs/hour × 10% miss rate × 6,000 files/miss = 108,000 files/hour

Compare your result against these thresholds:

Files imported per hour	Expected workspace impact
Below 150,000	Normal operation
150,000 – 300,000	Degraded performance. Some jobs may experience delays or failures.
Above 300,000	Jobs do not complete reliably.

Best practices

Standardize patterns

- **Do**: Publish three or fewer approved sparse patterns per repository. Shared patterns consolidate load and maximize cache hits.
- **Don't**: Allow custom per-team patterns. Even one extra folder creates a new cache entry and triggers a full re-import.

Manage commit churn

- **Do**: Point jobs at a stable release branch. Batch merges into scheduled release windows so multiple runs share the same cached commit.
- **Don't**: Use sparse checkouts with frequently updated branches like `master` or `main`. Because the cache is based on the exact commit hash, every new commit invalidates the cache and causes a full re-import for each job run.

Manage load

- **Do**: Remove large binaries, generated artifacts, and data files from source control to reduce the repository size unconditionally.
- **Don't**: Leave redundant jobs running at high frequency. Lower trigger frequency for jobs that don't require continuous execution, stagger schedules, or consolidate jobs that share the same checkout.

Manage commit churn with a release branch

When jobs target a fast-moving branch like `master` or `main`, the commit hash changes frequently, causing cache misses on nearly every run. Using a dedicated release branch that updates on a fixed schedule improves cache hit rates.

By pointing all jobs to an hourly release branch, all runs within that hour resolve to the same commit hash and share the same cache entry.

To configure a release branch:

1. Create a long-lived branch (for example, `release-candidate`) in your Git repository.
2. Automate updating this branch to match `master` on a fixed schedule, such as the top of every hour.
3. Configure your Git-backed jobs to use `release-candidate` as their target Git reference.

Review these trade-offs before implementing:

Consideration	Description
Commit lag	Jobs run against code up to one hour behind `master`. Acceptable for most batch workloads, but may not suit jobs requiring the latest commit.
Failure window	If the release cut job fails, the branch is not updated for that hour and jobs continue running against the previous commit. Databricks recommends alerting on the cut job.

Example: automate with GitHub Actions

The following GitHub Actions workflow automates an hourly release branch cut.

Step 1: Commit a `.github/workflows/cut-release-branch.yml` file to your repository:

YAML

```yaml
name: Cut Hourly Release Candidate
```

Ask Genie

```yaml
on:
  schedule:
    - cron: '0 * * * *'
  workflow_dispatch:

jobs:
  update-branch:
    runs-on: ubuntu-latest
    permissions:
      contents: write

    steps:
      - name: Checkout main branch
        uses: actions/checkout@v4
        with:
          ref: main
          fetch-depth: 0

      - name: Update release-candidate branch
        run: |
          git push origin HEAD:release-candidate --force
```

Step 2: Manually trigger the GitHub Action to verify that the `release-candidate` branch is created.

Step 3: Update your existing jobs to use `release-candidate` as the target Git reference.

Enable sparse checkout using the Jobs API

To enable sparse checkout, include a `sparse_checkout` block inside `git_source` when creating or updating a job:

JSON

```json
{
  "git_source": {
    "git_url": "https://github.com/example/my-repo",
    "git_provider": "gitHub",
    "git_branch": "release-candidate",
    "sparse_checkout": {
      "patterns": ["src/models", "src/utils"]
    }
```

```
    }
  }
```

Each string in `patterns` is a directory path relative to the repository root. All files within each specified directory are included in the checkout.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **Jan 23, 2026***

Jobs with a large number of tasks

You can create jobs that contain up to 1000 tasks. The following topics describe how to solve specific issues that may come up with large jobs that have more than 100 tasks.

Get an error that only 100 tasks are allowed

If you receive an error like `Resources with more than 100 tasks can only be handled by API 2.2 and above. Your resource has 200 tasks` then you must update your Databricks SDK or Databricks CLI.

The following are the minimum SDK versions required, by language:

SDK Language	Version
Go	0.60.0
Python	0.45.0
Java	0.42.0

For information on using and upgrading SDKs, see Databricks SDKs.

Your Databricks CLI must be at least version 0.244.0. For information about installing and updating the Databricks CLI, see Install or update the Databricks CLI.

Get an error that only 150 execution contexts are allowed

If you receive an error like `Too many execution contexts are open right now. (Limit set to 150)`, then your job is trying to run too many tasks on a single compute cluster. You must split the tasks across multiple clusters. For more information, see Configure compute for jobs.

The matrix view for job runs slows down or doesn't show individual tasks

If you are viewing a large number of tasks in the matrix view, for example, a job with 500 tasks, viewing 100 job runs, the UI may slow down. In extreme cases, the view may show only the summary for each job, rather than details by task.

You can filter your views to a shorter time span to see a smaller number of tasks at a single time. This can improve speed and allow the view to show details for all tasks.

For more information about the matrix view, see View runs for a single job.

*Last updated on **Jan 24, 2026***

Backfill jobs

After you have created a job that runs on a schedule, there are times when you need to backfill data into the system. For example:

- An error in the system caused data to be missed over a period of time. You need to backfill data for that date range.
- You want to bring in data from before the system was running for historical analysis.

A backfill for data should use the same job automation that you use to keep your data up to date. Job backfills allow you to run jobs using your existing automation for different date ranges.

How does a backfill work?

A backfill runs your job multiple times with different parameters to recreate the job runs from an earlier date range. To create a backfill, you select the date range that you want to backfill for and the time interval that is used to split the range into job runs. When you run the backfill, it generates the number of runs based on the date range and time interval selected.

You can set up the backfill to override the existing parameters that are passed to each run, or you can add new parameters that the job uses when present.

> **NOTE**
>
> The job must handle the parameters that you give it to process the correct data for the backfill date range. This might require changes to your job. For an example of how to create a query that correctly handles a date range, see the tutorial, Setup a recurring query with backfill support using Lakeflow Jobs.

How do date ranges generate job runs?

A backfill typically consists of multiple job runs set to the same time interval as your regular job runs over the time that you need to backfill. By using the same time interval your regular job, the performance and optimizations that you have set up in your job are maintained. Splitting the backfill into multiple runs also allows them to be triggered in parallel, completing the backfill faster.

For example, if you had errors in your hourly processing on August 9th and 10th, you need to generate a backfill to cover data that was missed on those days. Create a backfill for those two days, giving it a time interval of 1 hour. This triggers up to 48 runs, one for each hour's worth of data in the date range. Each run is passed parameters for the hour it should process.

How do backfills get run in parallel?

A backfill that has more than one time interval in its date range triggers multiple runs, which can be run in parallel in many cases. How can you set up parallel runs?

Jobs have a maximum concurrency limit. By default this is set to one, but you can set it higher. This is in the job settings, and can be accessed under **Advanced settings**:

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Click the name of the job for which you want to edit settings. This opens the job monitoring details page.
3. Settings are shown in the right panel. Select **Advanced settings**.
4. Under **Advanced settings**, you can edit the maximum concurrent runs by clicking .

   Setting this to a number greater than one allows that number of parallel job runs. For example, setting this to `2` allows two job runs to run at the same time.

> **NOTE**
>
> Jobs with pipeline tasks can't be run in parallel, and display a warning. See Limitations.

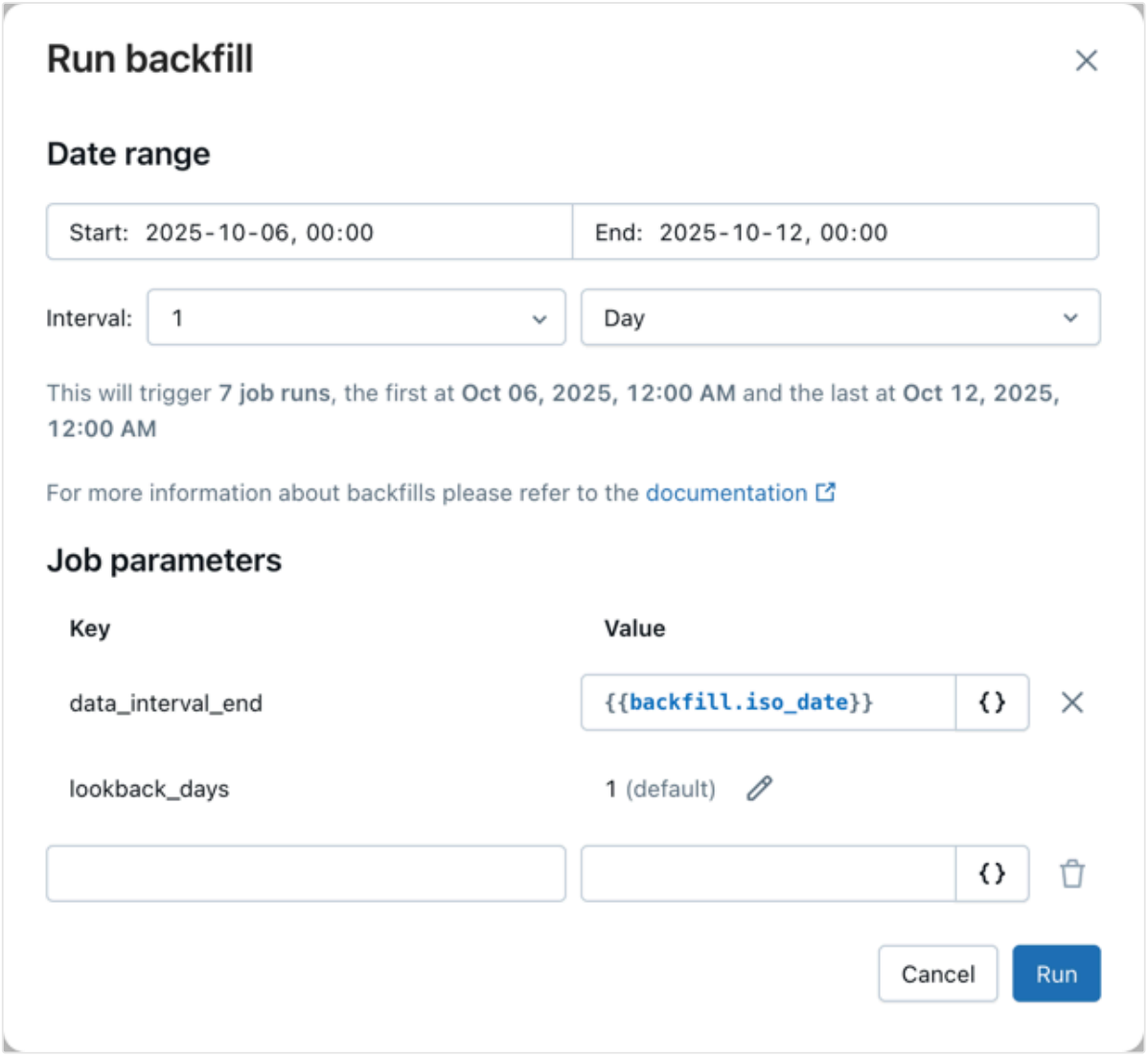
Prerequisites

Your job must support a parameter for a date or time used to get the correct data in the backfill process. Many jobs that run on a regular schedule already have a parameter that uses a time that you can override, but you can also set a parameter that is unique to your backfills.

How to create a backfill

You create a backfill through the jobs user interface.

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Click the name of the job for which you want to create a backfill. This opens the job monitoring details page.
3. Click the down arrow (re:[Chevron down icon]) next to **Run now** at the top of the page.
4. Select **Run backfill** from the drop down that appears. This opens the **Run backfill** dialog.



5. Select a **Date and time range** to the range that you want to backfill.

6. Databricks chooses a default time interval for your backfills, based on your trigger or schedule. If you want to change the time interval, in the drop downs to the right of **Every**, select the time interval you want. For example, to use 1 hour time interval, choose **1** and **Hour** for the time interval. At the bottom of the dialog, the interface includes a note of the number of new runs that your selections generate.

   If your backfill splits the date range into more than 100 runs, a warning appears when you are setting it up and you are unable to start the backfills. In this case, you must alter your **Date and time range** into multiple steps with under 100 runs each. You can do this with multiple backfills, or by increasing the time interval for the backfills.

7. Under **Job parameters**, your existing parameters are shown with keys and values. If you have a parameter or set of parameters for the date range of the job, you can

override those by editing the **Value** field. Otherwise you can create a new parameter by adding a new **Key** and **Value**.

For example, if your job takes the job trigger time as an iso datetime parameter, you could override that parameter with `{{backfill.iso_datetime}}`. To see a list of possible value references, click the **{ }** button next to the value you want to override, and select a reference to insert it into the **Value** field.

For a list of parameters, including the backfill-specific parameters, see Supported value references.

You can also add other parameters that aren't in your normal parameter list, that can be picked up by your automation. For example, if you wanted to have different processing for a backfill job, you could add a `backfill` parameter set to `true`.

8. Optionally override any other parameters that are used to control the job to match the data that you want to backfill.

9. Click **Run** to start the backfill runs.

Backfill runs show up in the runs list with the text `Backfill` in their name, to differentiate them from other job runs.

Limitations

- Backfills always run completely. You can't run a subset of tasks or tables in a backfill.

- Lakeflow Spark Declarative Pipelines specific limitations:

  - Pipeline tasks are not parameterized. The pipeline is run as-is.
  - Pipeline tasks can't be run concurrently, so jobs with pipeline tasks are run sequentially rather than in parallel. A warning is shown in the **Run backfill** dialog when this happens, and in **Settings**, if you set the maximum concurrent runs greater than 1 when there is a pipeline task in the job.

  Databricks recommends backfilling Lakeflow Spark Declarative Pipelines using the pipeline append once functionality. For more information, see Backfilling historical data with pipelines.

Next steps

To see a tutorial that shows how to create a query and parameters that can be used by a backfill job, see Setup a recurring query with backfill support using Lakeflow Jobs.

*Last updated on **Apr 30, 2026***

Configure and edit tasks in Lakeflow Jobs

This article focuses on instructions for creating, configuring, and editing tasks using the **Jobs & Pipelines** workspace UI.

Databricks manages tasks as components of Lakeflow Jobs. A job has one or more tasks. You create a new job in the workspace UI by configuring the first task. To configure a new job, see Configure and edit Lakeflow Jobs.

Each task has an associated compute resource that runs the task logic. If you are using serverless, Databricks configures your compute resources. If you are not using serverless, see Configure compute for jobs.

Databricks has other entry points and tools for task configuration, including the following:

- Jobs REST API reference
- Databricks CLI
- Create and manage scheduled notebook jobs

Create or configure a task

To edit an existing task or add a new task with the workspace UI, select an existing job using the following steps:

1. In your Databricks workspace's sidebar, click **Jobs & Pipelines**.
2. Optionally, select the **Jobs** and **Owned by me** filters.
3. Click your job's **Name** link.
4. Click the **Tasks** tab. The task graph appears.

5. To edit a task, click the task name. The task configuration appears below the task graph.
6. To add a task, click 

.

Types of tasks

Configuration options and instructions vary by task. The following task types are available:

- Notebook
- Python script
- Python wheel
- SQL
- Pipeline
- SQL Alert (Beta)
- Dashboards
- Power BI
- dbt
- dbt platform (Beta)
- JAR
- Spark Submit
- Run Job
- If/else
- For each

Clone a task

Clone tasks to copy all the configurations of an existing task, including upstream dependencies.

To clone a task, do the following:

1. Select the task in the task graph.

2. Click .
3. Specify a **Cloned task name** and click **Clone**.

Disable a task

Disable a task to skip it at run time without removing it from the job. The task keeps its configuration and run history, so you can re-enable it later without rebuilding the task.

Common scenarios for disabling a task include the following:

- Temporarily excluding a task while debugging an upstream issue without losing the task's settings.
- Pausing a broken task so the rest of the job continues to run on schedule.
- Keeping the job's Directed Acyclic Graph (DAG) and run history intact while deciding whether to remove a task.

To disable a task in a job:

1. Open the job and select the task in the DAG.
2. Click to disable the task.

To re-enable a disabled task, select the task and click .

To skip a task for a single run without changing the job settings, use Run a job with different settings instead.

For how disabled tasks affect downstream tasks, repairs, and partial runs, see Disabled tasks in Lakeflow Jobs.

Delete a task

To delete a task, do the following:

1. Select the task in the task graph.
2. Click and select **Delete task**.

To keep the task's configuration and run history instead of deleting it, disable the task.

Copy a task path

Certain task types, for example, notebook tasks, allow you to copy the path to the task source code:

1. Click the **Tasks** tab.
2. Select the task containing the path to copy.
3. Click  next to the task path to copy the path to the clipboard.

Advanced task settings

The following advanced settings control retries for failed tasks and timeout policies for unresponsive tasks.

> **NOTE**
>
> You can set notifications at the task or job level. See Add notifications on a job.

Set a retry policy

The default setting for task retries depends on the job configuration. For most configurations, the default setting does not retry any tasks on task failure.

Serverless jobs auto-optimize retries by default. See Configure serverless compute auto-optimization to disallow retries

Continuous jobs use an exponential backoff retry policy. See How are failures handled for continuous jobs?.

To configure a policy that determines when and how many times failed task runs are retried, click **+ Add** next to **Retries**.

The retry interval is calculated in milliseconds between the start of the failed run and the subsequent retry run.

> **NOTE**
>
> Ask Genie

If you configure both **Timeout** and **Retries**, the timeout applies to each retry.

Configure thresholds for task run duration or streaming backlog metrics

PREVIEW

Streaming observability for Lakeflow Jobs is in Public Preview.

You can configure optional thresholds for task run duration or streaming backlog metrics. To configure duration thresholds or streaming metric thresholds, click **Metric thresholds** in the task configuration panel.

To configure task duration thresholds, including expected and maximum completion times for the task, select **Run duration** in the **Metric** drop-down menu. Enter a duration in the **Warning** field to configure the tasks's expected completion time. If the task run exceeds this threshold, an event is triggered. To configure a maximum completion time for a task, enter the maximum duration in the **Timeout** field. If the task does not complete in this time, Databricks sets its status to "Timed Out".

To configure a threshold for a streaming backlog metric, select the metric in the **Metric** drop-down menu and enter a value for the threshold. To learn about the specific metrics supported by a streaming source, see View metrics for streaming tasks.

Enter a duration in the **Warning** field to configure the task's expected completion time. If the task exceeds this threshold, an event is triggered. You can use this event to notify when a task is running slowly. See Configure notifications for slow jobs.

To configure a maximum completion time for a task, enter the maximum duration in the **Timeout** field. If the task does not complete in this time, Databricks sets its status to "Timed Out".

If an event is triggered because a threshold is exceeded, you can use the event to send a notification. See Configure notifications for slow jobs.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **Apr 16, 2026***

Notebook task for jobs

Use the notebook task to deploy Databricks notebooks.

Configure a notebook task

Before you begin, you must have your notebook in a location accessible by the user configuring the job.

> **NOTE**
> The jobs UI displays options dynamically based on other configured settings.

To begin the flow to configure a `Notebook` task:

1. Navigate to the **Tasks** tab in the Jobs UI.
2. Click **Add task**.
3. Enter a name into the **Task name** field.
4. In the **Type** drop-down menu, select `Notebook`.

Configure the source

In the **Source** drop-down menu, select a location for the Python script using one of the following options.

Workspace

Use **Workspace** to configure a notebook stored in the workspace by completing the following steps:

1. Click the **Path** field. The **Select Notebook** dialog appears.

2. Browse to the notebook, click to highlight the file, and click **Confirm**.

> **NOTE**
>
> You can use this option to configure a task for a notebook stored in a Databricks Git folder. Databricks recommends using the **Git provider** option and a remote Git repository for versioning assets scheduled with jobs.

Git provider

Use **Git provider** to configure a notebook in a remote Git repository.

The options displayed by the UI depend on whether or not you have already configured a Git provider elsewhere. Only one remote Git repository can be used for all tasks in a job. See Use Git with Lakeflow Jobs.

> **IMPORTANT**
>
> Notebooks created by Lakeflow Jobs that run from remote Git repositories are ephemeral and cannot be relied upon to track MLflow runs, experiments, or models. When creating a notebook from a job, use a workspace MLflow experiment (instead of a notebook MLflow experiment) and call `mlflow.set_experiment("/path/to/experiment")` in the workspace notebook before running any MLflow tracking code. For more details, see Prevent data loss in MLflow experiments.

The **Path** field appears after you have configured a git reference.

Enter the relative path for your notebook, such as `etl/bronze/ingest.py`.

> **IMPORTANT**
>
> When you enter the relative path, don't begin with `/` or `./`. For example, if the absolute path for the notebook you want to access is `/etl/bronze/ingest.py`, enter `etl/bronze/ingest.py` in the **Path** field.

Configure compute and dependent libraries

1. Use **Compute** to select or configure a cluster that supports the logic in your notebook.
2. If you use `Serverless` compute, install libraries directly inside notebook, using the Environment panel or using `%pip install`. See Configure the serverless environment.
3. For all other compute configurations, click **+ Add** under **Dependent libraries**. The **Add dependent library** dialogue appears.
   - You can select an existing library or upload a new library.
   - You can only use libraries stored in a location supported by your compute configurations. See Python library support.
   - Each **Library Source** has a different flow for selecting or uploading a library. See Install libraries.

Finalize job configuration

1. (Optional) Configure **Parameters** as key-value pairs that can be accessed in the notebook using `dbutils.widgets`. See Configure task parameters.
2. Click **Save task**.

Limitations

Total notebook cell output (the combined output of all notebook cells) is subject to a 30MB size limit. Additionally, individual cell output is subject to an 8MB size limit. If total cell output exceeds 30MB in size, or if the output of an individual cell is larger than 8MB, the run is canceled and marked as failed.

If you need help finding cells near or beyond the limit, run the notebook against an all-purpose cluster and use this notebook autosave technique.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **Jan 24, 2026***

Clean Room notebook task for jobs

Use the Clean Room notebook task to run Databricks notebooks in a Clean Room as part of a workflow.

For more information about using Clean Room notebook tasks to create complex workflows, see Use Lakeflow Jobs to run clean room notebooks.

Before you begin

The principal who runs the task must have the `EXECUTE CLEAN ROOM TASK` privilege on the Clean Room that the notebook is in.

Configure a Clean Room notebook task

You can add a task to an existing workflow or create a new job that uses the Clean Room notebook task type.

> **NOTE**
> The jobs UI displays options dynamically based on other configured settings.

1. Navigate to the task edit page and select the Clean Room notebook task type:

   For a new job:

   i. In your workspace, click **Jobs & Pipelines** in the sidebar.
   ii. Under **New**, click **Job**.

iii. In the **Type** drop-down menu, select `Clean Room notebook`.

For an existing job:

i. In your Databricks workspace's sidebar, click **Jobs & Pipelines**.

ii. Optionally, select the **Jobs** and **Owned by me** filters.

iii. Select the job and go to the **Tasks** tab.

iv. Click **+ Add task** and select **Clean Room notebook**.

2. Select the **Clean Room** that contains the notebook.

3. Select the **Notebook**.

4. (Optional) If the notebook run depends on another task being completed, select the task dependencies from the **Depends on** drop-down menu.

5. (Optional) If the notebook uses `dbutils.widgets` to pass parameters, configure **Parameters** as key-value pairs that can be accessed by the notebook. See Configure task parameters.

6. (Optional) Configure a warning and timeout **Duration threshold**, **Notifications**, and a **Retries** policy.

7. Click **Save task**

8. Review the notebook and confirm that the notebook content is up to date.

   The notebook is displayed in preview mode.

9. Click **Continue**.

*Last updated on **Apr 16, 2026***

Python script task for jobs

Use the **Python script** task to run a Python file.

Configure a Python script task

Before you begin, you must upload your Python script to a location accessible to the user configuring the job. Databricks recommends using workspace files for Python scripts. See What are workspace files?.

> **NOTE**
>
> The jobs UI displays options dynamically based on other configured settings.
>
> Databricks recommends against storing code or data using DBFS root or mounts. Instead, you can migrate Python scripts to workspace files or volumes or use URIs to access cloud object storage.

To begin the flow to configure a `Python script` task:

1. Navigate to the **Tasks** tab in the Jobs UI.
2. Click **Add task**.
3. Enter a name into the **Task name** field.
4. In the **Type** drop-down menu, select `Python script`.

Configure the source

In the **Source** drop-down menu, select a location for the Python script using one of the following options.

Workspace

Use **Workspace** to configure a Python script stored using workspace files.

1. Click the **Path** field. The **Select Python File** dialog appears.
2. Browse to the Python script, click to highlight the file, and click **Confirm**.

> **NOTE**
>
> You can use this option to configure a task on a Python script stored in a Databricks Git folder. Databricks recommends using the **Git provider** option and a remote Git repository to version assets scheduled with jobs.

DBFS/S3

Use **DBFS/S3** to configure a Python script stored in a volume, cloud object storage location, or the DBFS root.

Databricks recommends storing Python scripts in Unity Catalog volumes or cloud object storage.

In the **Path** field, enter the URI to your Python script. For example, `dbfs:/path/to/script.py` or `s3://bucket-name/path/to/script.py`.

Git provider

Use **Git provider** to configure a Python script stored in a remote Git repository.

The options displayed by the UI depend on whether or not you have already configured a Git provider elsewhere. Only one remote Git repository can be used for all tasks in a job. See Use Git with Lakeflow Jobs.

The **Path** field appears after you have configured a git reference.

Enter the relative path for your Python script, such as `etl/bronze/ingest.py`.

> **IMPORTANT**
>
> When you enter the relative path, don't begin with `/` or `./`. For example, if the absolute path for the Python code you want to access is `/etl/bronze/ingest.py`, enter `etl/bronze/ingest.py` in the **Path** field.

Configure compute and dependent libraries

1. Use **Compute** to select or configure a cluster that supports the logic in your script.
2. If you use `Serverless` compute, use the **Environment and Libraries** field to select, edit, or add a new environment. See Configure the serverless environment.
3. For all other compute configurations, click **+ Add** under **Dependent libraries**. The **Add dependent library** dialogue appears.
   - You can select an existing library or upload a new library.
   - You can only use libraries stored in a location supported by your compute configurations. See Python library support.
   - Each **Library Source** has a different flow for selecting or uploading a library. See Install libraries.

Finalize job configuration

1. (Optional) Configure **Parameters** as a list of strings passed as CLI arguments to the Python script. See Configure task parameters.
2. Click **Save task**.

*Last updated on **Jan 23, 2026***

Python Wheel task for jobs

Use the Python wheel task type to deploy code packaged as a Python wheel.

Requirements

- You must upload your Python wheel file to a location or repository compatible with your compute configuration.
- You must know the name and entry point defined in the `setup.py` file.

Configure a Python wheel task

> **NOTE**
>
> The jobs UI displays options dynamically based on other configured settings.

Add a `Python wheel` task from the **Tasks** tab in the Jobs UI by doing the following:

1. Click **Add task**.
2. Enter a name into the **Task name** field.
3. In the **Type** drop-down menu, select `Python wheel`.
4. In the **Package name** field, enter the value assigned to the `name` variable in `setup.py`.
5. In the **Entry Point** field, enter the function that runs the logic in the wheel. This function must be defined as a key in the `entry_points` dictionary in `setup.py`.
6. Use **Compute** to select or configure a cluster that supports the logic in your wheel.
7. If you use `Serverless` compute, use the **Environment and Libraries** field to select, edit, or add a new environment. See Configure the serverless

8. For all other compute configurations, click **+ Add** under **Dependent libraries**. The **Add dependent library** dialogue appears.
    - You can select an existing Python wheel or upload a new Python wheel file.
    - You must store wheel files in a location supported by your compute configurations. See Python library support.
    - Each **Library Source** has a different flow for selecting or uploading a wheel. See Install libraries.
9. (Optional) Configure **Parameters**. You can use either **Positional arguments** or **Keyword arguments**. See Configure task parameters.
10. Click **Save task**.

*Last updated on **Apr 16, 2026***

SQL task for jobs

Use the SQL task type to configure a SQL query, alert, or SQL file.

The **SQL** task requires Databricks SQL and a serverless or pro SQL warehouse.

Configure a SQL task

Before you begin, you must have your SQL asset in a location accessible by the user configuring the job.

> **NOTE**
>
> The jobs UI displays options dynamically based on other configured settings.

To begin the flow to configure a `SQL` task:

1. Navigate to the **Tasks** tab in the Jobs UI.
2. In the **Type** drop-down menu, select `SQL`.

Configure the SQL task type

In the **SQL task** drop-down menu, select a SQL task type using one of the following options.

Query

Select a **SQL query** to run against the specified **SQL warehouse**.

Alert

Select a **SQL alert** to evaluate using the specified **SQL warehouse**.

(Optional) Add **Subscribers** to notify with the results of the alert.

File

Use **File** to run a `.sql` file using the specified **SQL warehouse**.

The file can contain multiple SQL statements separated by semicolons (`;`).

You must configure a **Source** for the SQL file using one of the following options.

Workspace

Use **Workspace** to configure a SQL file stored as a workspace file.

1. Click the **Path** field. The **Select SQL file** dialog appears.
2. Browse to the SQL file, click to highlight the file, and click **Confirm**.

> **NOTE**
>
> You can use this option to configure a task for a SQL file stored in a Databricks Git folder. Databricks recommends using the **Git provider** option with a remote Git repository to version assets scheduled with jobs.

Git provider

Use **Git provider** to configure a SQL file stored in a remote Git repository.

The options displayed by the UI depend on whether or not you have already configured a Git provider elsewhere. Only one remote Git repository can be used for all tasks in a job. See Use Git with Lakeflow Jobs.

The **Path** field appears after you have configured a git reference.

Enter the relative path for your notebook, such as `etl/bronze/ingest.sql`.

> **IMPORTANT**

When you enter the relative path, don't begin with `/` or `./`. For example, if the absolute path for the notebook you want to access is `/etl/bronze/ingest.sql`, enter `etl/bronze/ingest.sql` in the **Path** field.

Finalize job configuration

1. (Optional) Configure **Parameters** as key-value pairs referenced in the configured SQL asset. Alerts do not support parameters. See Configure task parameters.
2. Click **Save task**.

*Last updated on **Jan 23, 2026***

Pipeline task for jobs

Lakeflow Jobs provide a procedural approach to defining relationships between *tasks*. Lakeflow Spark Declarative Pipelines provide a declarative approach to defining relationships between *datasets* and *transformations*. This page describes how you can schedule triggered Lakeflow Spark Declarative Pipelines to run as a task in a job, using the Jobs UI, the Lakeflow Spark Declarative Pipelines UI, or SQL.

> **NOTE**
>
> A *triggered* pipeline is a pipeline that does not run continuously, but must be triggered to start. A pipeline task can be the triggering mechanism for a triggered pipeline. Continuous pipelines do not need to be triggered, so triggering them through a task would be redundant. To learn more about triggered and continuous pipelines, see Triggered vs. continuous pipeline mode.

Configure a pipeline task with the Jobs UI

Lakeflow Spark Declarative Pipelines manage all configurations for source code and compute in the pipeline definition.

To add a pipeline to a job, complete the following steps:

1. Create and name a new task and select **pipeline** for the **Type**.
2. In the **Pipeline** drop-down menu, select an existing pipeline. The pipeline must be a triggered pipeline. Continuous pipelines are not supported as a job task.
3. You can optionally trigger a full refresh on the pipeline.

> **NOTE**

You can also create a new ingestion pipeline when creating a task by choosing **+ New ingestion pipeline** from the **Add Task** pane or the tasks **Type** drop-down.

Schedule a pipeline with the pipeline UI

Adding a schedule to a pipeline creates a job with a single pipeline task. You can only configure time-based schedule triggers using this UI. For more advanced triggering options, see Configure a pipeline task with the Jobs UI.

Configure a pipeline task in a scheduled job using the pipeline UI by completing the following steps:

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Click on the pipeline **Name**. The pipeline UI appears.
3. Click **Schedule**.
   - If no schedule exists for the pipeline, the **New schedule** dialog appears.
   - If one or more schedules already exist, click **Add schedule**.
4. Enter a unique name for the job in the **Job name** field.
5. (Optional) Update the schedule frequency.
   - Select **Advanced** for more verbose options including cron syntax.
6. (Optional) Under **More options**, configure one or more email addresses to receive alerts on pipeline start, success, or failure.
7. Click **Create**.

NOTE

If the pipeline is included in one or more scheduled jobs, the **Schedule** button shows the number of existing schedules, for example, **Schedule (5)**.

Add a schedule to a materialized view or streaming table in Databricks SQL

Materialized views and streaming tables defined in Databricks SQL support time-based scheduling specified in `CREATE` or `ALTER` commands.

For details, see the following articles:

- CREATE MATERIALIZED VIEW
- CREATE STREAMING TABLE
- ALTER MATERIALIZED VIEW
- ALTER STREAMING TABLE

*Last updated on **Mar 5, 2026***

SQL alert task for jobs

> **BETA**
>
> This feature is in Beta. Workspace admins can control access to this feature from the **Previews** page. See Manage Databricks previews.

Use a SQL alert task to evaluate a Databricks SQL alert as part of a job. SQL alert tasks allow you to integrate alert-based monitoring into your data pipelines, enabling automated condition checks and notifications alongside other job tasks. To learn more about Databricks SQL alerts, see Databricks SQL alerts.

Prerequisites

To use the SQL alert task, you must meet the following prerequisites:

- A workspace admin must enable the preview. See Manage Databricks previews.
- You must have an existing Databricks SQL alert in your workspace. To create an alert, see Create an alert.
- You must have at least CAN RUN permission on the alert you want to use.
- You must have access to a serverless or pro SQL warehouse.

Configure a SQL alert task

The jobs UI displays options dynamically based on other configured settings. To configure a `SQL Alert` task:

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Click **Create**, then **Job**.
3. Click **Add another task type**. Search for **SQL Alert** and click the tile.
4. Enter a **Task name**.

5. In the **Alert** drop-down menu, select the Databricks SQL alert that you want to evaluate.
6. (Optional) In the **SQL warehouse** drop-down menu, select the SQL warehouse to use for running the alert query. If not set, the alert's internal warehouse is used. You must use a serverless or pro SQL warehouse for SQL alert tasks.
7. (Optional) In the **Subscribers** drop-down, choose the users and notification destinations that should receive notifications with the alert results. If not set, the alert's internal subscribers (if applicable) receive notifications.
8. (Optional) To configure **Duration threshold**, **Notifications**, or **Retries**, see Advanced task settings.
9. Click **Save task**.

SQL alert task behavior

When a job runs a SQL alert task, the following occurs:

1. The alert's query executes on the specified SQL warehouse.
2. The alert condition is evaluated against the query results.
3. Subscribers configured on the task receive notifications based on the alert outcome.

The SQL alert task reports one of the following statuses:

- **Succeeded**: The alert was evaluated successfully, regardless of whether the alert condition was triggered or not.
- **Failed**: An error occurred during alert evaluation, such as a SQL warehouse connectivity issue or a query error.

> **NOTE**
> The SQL alert task evaluates the alert independently from any schedule configured on the alert itself. Schedules on the underlying alert are not affected by the job run.

Limitations

- SQL alert tasks do not support parameters. If you need to use parameterized queries, consider using a SQL task instead.
- During the Beta period, SQL alert tasks support only Databricks SQL alerts. Legacy alerts are not supported.

Last updated on ***Feb 23, 2026***

Dashboard task for jobs

Use a dashboard task to refresh results on a published Databricks dashboard as part of a job.

Configure a dashboard task

Before you begin, make sure the dashboard is published and accessible to the user configuring the job. You must have at least CAN VIEW access to the dashboard you want to deploy.

> **NOTE**
> The jobs UI displays options dynamically based on other configured settings.

To begin the flow to configure a `Dashboard` task:

1. Navigate to the **Tasks** tab in the Jobs UI. For general instructions, see Create or configure a task.
2. Enter a **Task name**.
3. In the **Type** drop-down menu, select `Dashboard`.
4. In the **Dashboard** drop-down menu, select the dashboard that you want to update.
5. (Optional) Choose the SQL warehouse used to refresh the dashboard. If not specified, the task uses the dashboard's default warehouse.

> **NOTE**
> You must use a serverless or pro SQL warehouse for dashboard tasks.

1. In the **Subscribers** drop-down, choose the users and notification destinations that should receive email updates when the dashboard is refreshed

separate from any subscribers configured through the Dashboards UI or API.

2. (Optional) To configure **Dashboard filters**, **Duration threshold**, **Notifications**, or **Retries**, see Advanced task settings.
3. Click **Save task**.

Dashboard execution and access

Dashboard access and caching behavior depend on how the dashboard is published:

- For dashboards published with shared data permissions, viewers access data and compute according to the dashboard publisher's credentials.
- For dashboards published without shared data permissions, the identity defined in the job's **Run as** setting is used to refresh the dashboard. The **Run as** setting defaults to the job owner. To change the **Run as** identity, see Configure the Run as user for job runs. Viewers use their own data and compute permissions to access the dashboard output and cannot view refreshed results generated by the **Run as** identity.

> **NOTE**
>
> Dashboards with shared data permissions are always updated using the publisher's credentials. You cannot use the **Run as** setting in the Jobs UI to change the **Run as** identity.

To learn more about publish settings for dashboards, see Publish a dashboard. To learn more about caching behavior for each access type, see Dataset optimization and caching.

*Last updated on **Jan 24, 2026***

Power BI task for jobs

> **PREVIEW**
>
> The Power BI task feature is in Public Preview.

While you can publish to Microsoft Power BI online manually from your Databricks workspace, you can use a Power BI task to orchestrate your Power BI semantic models automatically.

To learn more about publishing to Power BI in the Databricks UI, see Publish to the Power BI service from Databricks.

Before you begin

- You must follow the same requirements as when publishing to Power BI manually. For details, see Publish to Power BI Online from Databricks.
- Have, or create, a Power BI connection. See Create a Power BI connection in Unity Catalog for orchestration.
- You must have the `USE CONNECTION` privilege in Unity Catalog for that connection, as well as privileges for accessing the tables and SQL warehouse to be used. See Manage identities, permissions, and privileges for Lakeflow Jobs.

Configure a Power BI task

After you have a Power BI connection configured, you can create a task to automate publishing using that connection.

> **NOTE**
>
> The jobs UI displays options dynamically based on other configured s

To begin the flow to configure a `Power BI` task:

1. Navigate to the **Tasks** tab in the Jobs UI, for the job to which you want to add a task.
2. Click **+ Add task**.
3. Enter a **Task name**.
4. In the **Type** drop-down menu, select **Power BI**.
5. Configure the task properties (see the following table for the properties and their use).
6. Click **Save task**.

> **NOTE**
>
> Databricks recommends setting a Databricks service principal to be the **Run as** identity on the task. For the best practices, see Best practices for jobs governance. The service principal will require the necessary privileges to access the Databricks tables, schemas, Power BI connection, and SQL warehouse used by the task.
>
> When editing a task, the credentials of the current user are used, but when the task is run, the **Run as** identity is used. The identity must have the correct privileges to run the task.

Power BI task property	Description
SQL Warehouse	The SQL warehouse that processes refreshes in **Import** mode, or queries in **DirectQuery** mode for the semantic model.
Power BI connection	The Power BI connection for this task. The task uses this connection to fetch Power BI workspaces and semantic models, and to publish to Power BI. See Create a Power BI connection in Unity Catalog for orchestration.
Power BI workspace	The Power BI workspace to which a semantic model is published.

Power BI task property	Description
Power BI semantic model	The Power BI semantic model to publish. Select an existing model, or type a new model name and click **Publish new semantic model <name>**.
Overwrite existing model	By default, metadata updates are only appended to an existing model. Checking this box ensures all metadata and data updates propagate to Power BI semantic models when the task is run.
Power BI query mode	The default query mode for the tables being published. When **DirectQuery** is selected, you can also set query modes on individual tables using the **Configure table query modes** property. One of the following values: • **Import** The data for the model is cached in Power BI. To update the data, users must refresh the model before using it, which queries the SQL warehouse and loads the data. • **DirectQuery** The data is not stored in Power BI. When the user creates or loads a dashboard, the SQL warehouse is queried for the most recent data. Power BI query mode is also referred to as storage mode in Power BI. For more information about query modes, see Semantic model modes in the Power BI service.
Tables to update	The source tables and schemas for the semantic model. If you select a schema for this property, then when you run the task, all tables under the schema at that time are used for the update. The task updates any new tables, columns, comments, and primary key/foreign key relationships.
Authentication method	Defines how the semantic model authenticates back to the chosen SQL warehouse. When using OAuth, credentials may need to be configured on Power BI after the first task run.

Power BI task property	Description
	When using PAT, it will generate and embed a PAT for the **Run as** identity.
Configure table query modes	When **DirectQuery** is selected as the **Power BI query mode**, you can optionally set individual tables to use **Dual** storage mode. Tables set to **Dual** storage mode can act as either **Import** or **DirectQuery** modes depending on the context of the query. For more information, see Semantic model modes in the Power BI service.
Refresh after update	This option is available if **Import** is selected as the query mode. By default, only the model metadata is updated, but if this checkbox is checked, then it will also trigger a data refresh (which will query the SQL warehouse). This refresh can be seen in the refresh history on Power BI.

Troubleshooting

For information about troubleshooting publishing to Power BI, see Power BI troubleshooting. If you are still having trouble, you can submit product feedback. See Submit product feedback.

*Last updated on **Apr 16, 2026***

dbt task for jobs

Use the dbt task to configure and run dbt projects on Databricks.

> **IMPORTANT**
>
> When dbt tasks run, Databricks injects the `DBT_ACCESS_TOKEN` for the principal configured in the **Run As** field.

Configure a dbt task

Add a `dbt` task from the **Tasks** tab in the Jobs UI by doing the following:

1. Click **Add task**.
2. Enter a name into the **Task name** field.
3. In the **Type** drop-down menu, select `dbt`.
4. In the **Source** drop-down menu, you can select **Workspace** to use a dbt project located in a Databricks workspace folder or **Git provider** for a project located in a remote Git repository.
   - If you select **Workspace**, use the provided file navigator to select the **Project directory**.
   - If you select **Git provider**, click **Edit** to enter Git information for the project repository. See Use Git with Lakeflow Jobs.

     If your project is not in the repo's root directory, use the **Project directory** field to specify the path to it.
5. The **dbt commands** text boxes default to the commands **dbt deps** and **dbt run**. The provided commands run in sequential order. Add, remove, or edit

these fields as necessary for your workflow. See What are dbt commands?.

6. In **SQL warehouse**, select a SQL warehouse to run the SQL generated by dbt. The **SQL warehouse** drop-down menu shows only serverless and pro SQL warehouses.

7. Specify a **Warehouse catalog**. If unset, the workspace default is used.

8. Specify a **Warehouse schema**. By default, the schema `default` is used.

9. Choose **dbt CLI compute** to run dbt Core. Databricks recommends using serverless compute for jobs or classic jobs compute configured with a single-node cluster.

10. Specify a `dbt-databricks` version for the task.

    If you use `Serverless` compute, use the **Environment and Libraries** field to select, edit, or add a new environment. See Configure the serverless environment.

    For all other compute configurations, the **Dependent libraries** field populates to `dbt-databricks>=1.0.0,<2.0.0` by default. Delete this setting and **+ Add** a **PyPi** library to pin a version.

    > **NOTE**
    > Databricks recommends pinning your dbt tasks to a specific version of the dbt-databricks package to ensure the same version is used for development and production runs. Databricks recommends version 1.6.0 or greater of the dbt-databricks package.

11. Click **Create task**.

What are dbt commands?

The **dbt commands** field allows you to specify commands to run using the dbt command line interface (CLI). For full details on the dbt CLI, see the dbt documentation.

Check the dbt documentation for commands supported by the specified version of dbt.

Pass options to dbt commands

The dbt node selection syntax lets you specify resources to include or exclude in a particular run. Commands such as `run` and `build` accept flags including `--select` and `--exclude`. See the dbt syntax overview docs for a complete description.

Additional configuration flags control how dbt runs your project. See the **Command line options** column in the official dbt docs for a list of available flags.

Some flags take positional arguments. Some arguments for flags are strings. Refer to the dbt documentation for examples and explanations.

Pass variables to dbt commands

Use the `--vars` flag to pass static or dynamic values to commands in **dbt commands** fields.

You pass a single-quote delimited JSON to `--vars`. All keys and values in the JSON must be double-quote delimited, as in the following example:

```
dbt run --vars '{"volume_path": "/Volumes/path/to/data", "date":
"2024/08/16"}'
```

Examples of parameterized dbt commands

You can reference task values, job parameters, and dynamic job parameters when working with dbt. Values are substituted as plain text into the **dbt commands** field before the command runs. For information about passing values between tasks or referencing jobs metadata, see Parameterize jobs.

These examples assume the following job parameters have been configured:

Parameter name	Parameter value
`volume_path`	`/Volumes/path/to/data`
`table_name`	`my_table`
`select_clause`	`--select "tag:nightly"`
`dbt_refresh`	`--full-refresh`

The following examples show valid ways to reference these parameters:

```
dbt run '{"volume_path": "{{job.parameters.volume_path}}"}'
dbt run --select "{{job.parameters.table_name}}"
dbt run {{job.parameters.select_clause}}
dbt run {{job.parameters.dbt_refresh}}
dbt run '{"volume_path": "{{job.parameters.volume_path}}"}'
{{job.parameters.dbt_refresh}}
```

You can also reference dynamic parameters and task values, as in the following examples:

```
dbt run --vars '{"date": "{{job.start_time.iso_date}}"}'
dbt run --vars '{"sales_count": "
{{tasks.sales_task.values.sales_count}}"}'
```

*Last updated on **Jan 23, 2026***

dbt platform task for jobs

> **BETA**
>
> This feature is in Beta. Workspace admins can control access to this feature from the **Previews** page. See Manage Databricks previews.

Use the dbt platform task to orchestrate and monitor existing dbt platform jobs directly from Databricks. This page explains how to select and trigger dbt jobs, set auto-retry options for failures, and monitor runs.

Differences between dbt platform and dbt tasks

Jobs offers two task types for dbt projects. Choose the right one based on where your dbt project is managed:

dbt platform task: Use this to orchestrate pre-existing dbt platform jobs. It connects to the dbt platform API and triggers a run there. Choose this if you want to centralize orchestration in Databricks while retaining all dbt platform benefits, such as monitoring and scheduling.

dbt task: Use this to run dbt core projects on a Databricks cluster with code from Git. Choose this if you need full control over the execution environment and prefer to manage dependencies entirely within Databricks. See dbt task for jobs.

Prerequisites

To use the dbt platform task, you must meet the following prerequisites:

- A workspace admin must enable the preview. See Manage Databrick

- You must have `CREATE CONNECTION` privileges on the Unity Catalog metastore in your workspace.
- Access to an existing dbt project with a defined job in the dbt platform. To learn more, see Jobs in the dbt platform in the dbt documentation.
- Permissions to generate a service token in the dbt platform. To learn more, see Service account tokens.

> **NOTE**
> For security and operational stability, Databricks recommends generating a service account token, not a personal access token. Service account tokens aren't tied to an individual user and can be easily scoped to provide the minimum necessary permissions.

Gather dbt platform details

To integrate dbt with Databricks, you need the following three details:

- Your dbt platform Account ID.
- An API key generated in the dbt platform.
- Your dbt platform deployment host URL.

The following sections describe how to find this required information.

Get your account ID:

To retrieve your account ID:

1. Log in to the dbt platform.
2. Navigate to **Settings** > **Account Settings**.
3. Get the Account ID from the URL suffix, which is in the following format: `https://cloud.getdbt.com/settings/accounts/{account_id}`.

Get your API key

To retrieve your API key:

1. Log in to the dbt platform.
2. Navigate to **Settings** > **Profile Setting** > **Your Profile** > **Access API** > **API Key**.

Host URL

Your host URL depends on your location and tenancy. See Access, Regions, & IP addresses in the dbt documentation to find the URL for your region.

Identify your region and tenancy (Multi-tenant or Cell-based). Use the **Access URL** column to get your host URL.

Tenancy Type	Region Example	Host URL Example
Multi-tenant	North America	`https://cloud.getdbt.com`
Cell-based	North America (`us-east-1`)	`https://12345.us1.dbt.com` (using `12345` as the Account ID)

dbt platform connection setup

Use the following steps to set up your dbt platform connection in Databricks.

1. Click **Catalog** in the sidebar.
2. Click + the plus icon in the schema browser. Then, click **Create a connection**. The **Set up connection** form opens.
3. Enter the following information, then click **Next**:
   - In **Connection name**, enter a name.
   - For **Connection type**, choose **dbt platform**.
4. Enter your dbt platform host URL in the **Host** text field. Do not include a trailing slash (`/`).
5. Enter your dbt platform Account ID and the API Token you collected in a previous step.
6. Click **Create connection** to confirm the connection details.

7. (Optional) Grant other users privileges to use the connection:
    - Choose the user IDs and groups you want to grant privileges to in the **Principals** drop-down menu.
    - Select the privileges you want to grant.
    - Click **Confirm**.

Create a new job with a dbt platform task

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Click **Create**, then **Job**. The new job is automatically named with an associated timestamp.
3. (Optional) Click the job name and enter a new name to edit it.
4. Click **Add another task type**. Search for dbt platform and click the tile to select it.
5. Enter a **Task name**.
6. Use the **dbt platform connection** drop-down menu to select the connection created previously.
7. Use the **dbt platform job** drop-down menu to select the dbt platform job that you want to orchestrate.
8. Click **Save task**.
9. (Optional) Click **Run now** to manually test your job.

Set a schedule or trigger

You can configure jobs to automatically trigger according to a time-based schedule or the arrival of new data. To learn more about the available options, see Automating jobs with schedules and triggers.

> **NOTE**
> Continuous triggers are not supported for dbt platform jobs.

Monitor runs

You can monitor Lakeflow jobs in the Databricks UI. For dbt platform jobs, you can also open a link that points to job run details in the dbt platform.

To monitor a run:

1. Click **Jobs & Pipelines** in the workspace sidebar.
2. (Optional) Select the **Jobs** and **Owned by me** filters.
3. Click your job's **Name** link.

   The **Runs** tab appears, showing matrix and list views of active and completed runs.
4. Click the link for the run in the **Start time** column in the runs list view. The dbt platform job status opens.
5. Click **View in dbt** to see the job run details in the dbt platform.

Last updated on ***Apr 10, 2026***

JAR task for jobs

Use the JAR task to deploy Scala or Java code compiled into a JAR (Java ARchive). Before you create a JAR task, see Create a Databricks compatible JAR.

> **PREVIEW**
>
> Serverless Scala and Java jobs are in Public Preview. You can use JAR tasks to deploy your JAR. See Manage Databricks previews if it's not already enabled.

Requirements

- Choose a compute configuration that supports your workload.
- Upload your JAR file to a location or Maven repository compatible with your compute. See JAR library support.
- **For standard access mode:** An admin must add Maven coordinates and JAR paths to an allowlist. See standard access limitations.

Configure a JAR task

Add a `JAR` task from the **Tasks** tab in the Jobs UI by doing the following:

1. Click **Add task**.
2. Enter a name into the **Task name** field.
3. In the **Type** drop-down menu, select `JAR`.
4. Specify the **Main class**.
   - This is the full name of the class containing the main method to
     This class must be included in a JAR configured as a **Dependent**

5. Click **Compute** to select or configure compute. Choose a classic or serverless compute.

6. Configure your environment and add dependencies:

   - For classic compute, click + **Add** under **Dependent libraries**. The **Add dependent library** dialog appears.

     - You can select an existing JAR file or upload a new JAR file.
     - Not all locations support JAR files.
     - Not all compute configurations support JAR files in all supported locations.
     - Each **Library Source** has a different flow for selecting or uploading a JAR file. See Install libraries.

   - For serverless compute, choose an environment then click edit to configure it.

     - You must select **4** or higher for the **Environment version**.
     - Add your JAR file.
     - Add any other dependencies that you have. Don't include Spark dependencies, because these are already provided in the environment by Databricks Connect. For more information on dependencies in JARs, see Create a Databricks compatible JAR.

7. (Optional) Configure **Parameters** as a list of strings passed as arguments to the main class. See Configure task parameters.

8. Click **Save task**.

*Last updated on **Jan 23, 2026***

Spark Submit Task Deprecation Notice & Migration Guide

WARNING

The **Spark Submit** task is deprecated and pending removal. Usage of this task type is disallowed for new use cases and strongly discouraged for existing customers. See Spark Submit (legacy) for the original documentation for this task type. Keep reading for migration instructions.

Why is Spark Submit being deprecated?

The **Spark Submit** task type is being deprecated due to technical limitations and feature gaps that are not in the JAR, Notebook, or Python script tasks. These tasks offer better integration with Databricks features, improved performance, and greater reliability.

Deprecation measures

Databricks is implementing the following measures in connection with the deprecation:

- **Restricted creation**: Only users who have used **Spark Submit** tasks in the preceding month, starting in November 2025, can create new **Spark Submit** tasks. If you need an exception, contact your account support.
- **Databricks Runtime version restrictions**: **Spark Submit** usage is r
existing Databricks Runtime versions and maintenance releases. Exis

Databricks Runtime versions with **Spark Submit** will continue to receive security and bugfix maintenance releases until the feature is shut down completely. Databricks Runtime 17.3+ and 18.x+ will not support this task type.

- **UI warnings**: Warnings appear throughout the Databricks UI where **Spark Submit** tasks are in use, and communications are sent to workspace administrators in accounts of existing users.

Migrate JVM workloads to JAR tasks

For JVM workloads, migrate your **Spark Submit** tasks to JAR tasks. JAR tasks provide better feature support and integration with Databricks.

Follow these steps to migrate:

1. Create a new JAR task in your job.
2. From your **Spark Submit** task parameters, identify the first three arguments. They generally follow this pattern: `["--class", "org.apache.spark.mainClassName", "dbfs:/path/to/jar_file.jar"]`
3. Remove the `--class` parameter.
4. Set the main class name (for example, `org.apache.spark.mainClassName`) as the **Main class** for your JAR task.
5. Provide the path to your JAR file (for example, `dbfs:/path/to/jar_file.jar`) in the JAR task configuration.
6. Copy any remaining arguments from your **Spark Submit** task to the JAR task parameters.
7. Run the JAR task and verify it works as expected.

For detailed information on configuring JAR tasks, see JAR task.

Migrate R workloads

If you're launching an R script directly from a **Spark Submit** task, multiple migration paths are available.

Option A: Use Notebook tasks

Migrate your R script to a Databricks notebook. Notebook tasks support a full set of features, including cluster autoscaling, and provide better integration with the Databricks platform.

Option B: Bootstrap R scripts from a Notebook task

Use a Notebook task to bootstrap your R scripts. Create a notebook with the following code and reference your R file as a job parameter. Modify to add parameters used by your R script, if needed:

R

```r
dbutils.widgets.text("script_path", "", "Path to script")
script_path <- dbutils.widgets.get("script_path")
source(script_path)
```

Find jobs that use Spark Submit tasks

You can use the following Python scripts to identify jobs in your workspace that contain Spark Submit tasks. A valid personal access or other token will be needed and your workspace URL should be used.

Option A: Fast scan (run this first, persistent jobs only)

This script only scans persistent jobs (created via `/jobs/create` or the web interface) and does not include ephemeral jobs created via `/runs/submit`. This is the recommended first-line method for identifying Spark Submit usage because it is much faster.

Python

```python
#!/usr/bin/env python3
"""
Requirements:
```

Ask Genie

```
    databricks-sdk>=0.20.0

Usage:
    export DATABRICKS_HOST="https://your-workspace.cloud.databricks.com"
    export DATABRICKS_TOKEN="your-token"
    python3 list_spark_submit_jobs.py

Output:
    CSV format with columns: Job ID, Owner ID/Email, Job Name

Incorrect:
    export DATABRICKS_HOST="https://your-workspace.cloud.databricks.com/?
o=12345678910"
"""

import csv
import os
import sys
from databricks.sdk import WorkspaceClient
from databricks.sdk.errors import PermissionDenied


def main():
    # Get credentials from environment
    workspace_url = os.environ.get("DATABRICKS_HOST")
    token = os.environ.get("DATABRICKS_TOKEN")

    if not workspace_url or not token:
        print(
            "Error: Set DATABRICKS_HOST and DATABRICKS_TOKEN environment
variables",
            file=sys.stderr,
        )
        sys.exit(1)

    # Initialize client
    client = WorkspaceClient(host=workspace_url, token=token)

    # Scan workspace for persistent jobs with Spark Submit tasks
    # Using list() to scan only persistent jobs (faster than list_runs())
    print(
        "Scanning workspace for persistent jobs with Spark Submit
tasks...",
        file=sys.stderr,
    )
    jobs_with_spark_submit = []
    total_jobs = 0

    # Iterate through all jobs (pagination is handled automatically by
the SDK)
```

```
    skipped_jobs = 0
    for job in client.jobs.list(expand_tasks=True, limit=25):
        try:
            total_jobs += 1
            if total_jobs % 1000 == 0:
                print(f"Scanned {total_jobs} jobs total",
file=sys.stderr)

            # Check if job has any Spark Submit tasks
            if job.settings and job.settings.tasks:
                has_spark_submit = any(
                    task.spark_submit_task is not None for task in
job.settings.tasks
                )

                if has_spark_submit:
                    # Extract job information
                    job_id = job.job_id
                    owner_email = job.creator_user_name or "Unknown"
                    job_name = job.settings.name or f"Job {job_id}"

                    jobs_with_spark_submit.append(
                        {"job_id": job_id, "owner_email": owner_email,
"job_name": job_name}
                    )
        except PermissionDenied:
            # Skip jobs that the user doesn't have permission to access
            skipped_jobs += 1
            continue

    # Print summary to stderr
    print(f"Scanned {total_jobs} jobs total", file=sys.stderr)
    if skipped_jobs > 0:
        print(
            f"Skipped {skipped_jobs} jobs due to insufficient
permissions",
            file=sys.stderr,
        )
    print(
        f"Found {len(jobs_with_spark_submit)} jobs with Spark Submit
tasks",
        file=sys.stderr,
    )
    print("", file=sys.stderr)

    # Output CSV to stdout
    if jobs_with_spark_submit:
        writer = csv.DictWriter(
            sys.stdout,
            fieldnames=["job_id", "owner_email", "job_name"],
```

```
            quoting=csv.QUOTE_MINIMAL,
        )
        writer.writeheader()
        writer.writerows(jobs_with_spark_submit)
    else:
        print("No jobs with Spark Submit tasks found.", file=sys.stderr)


if __name__ == "__main__":
    main()
```

Option B: Comprehensive scan (slower, includes ephemeral jobs from last 30 days)

If you need to identify ephemeral jobs created via `/runs/submit`, use this more exhaustive script. This script scans all job runs from the last 30 days in your workspace, including both persistent jobs (created via `/jobs/create`) and ephemeral jobs. This script may take hours to run in large workspaces.

Python

```
#!/usr/bin/env python3
"""
Requirements:
    databricks-sdk>=0.20.0

Usage:
    export DATABRICKS_HOST="https://your-workspace.cloud.databricks.com"
    export DATABRICKS_TOKEN="your-token"
    python3 list_spark_submit_runs.py

Output:
    CSV format with columns: Job ID, Run ID, Owner ID/Email, Job/Run Name

Incorrect:
    export DATABRICKS_HOST="https://your-workspace.cloud.databricks.com/?
o=12345678910"
"""

import csv
import os
import sys
import time
from databricks.sdk import WorkspaceClient
```

```python
from databricks.sdk.errors import PermissionDenied


def main():
    # Get credentials from environment
    workspace_url = os.environ.get("DATABRICKS_HOST")
    token = os.environ.get("DATABRICKS_TOKEN")

    if not workspace_url or not token:
        print(
            "Error: Set DATABRICKS_HOST and DATABRICKS_TOKEN environment
variables",
            file=sys.stderr,
        )
        sys.exit(1)

    # Initialize client
    client = WorkspaceClient(host=workspace_url, token=token)

    thirty_days_ago_ms = int((time.time() - 30 * 24 * 60 * 60) * 1000)

    # Scan workspace for runs with Spark Submit tasks
    # Using list_runs() instead of list() to include ephemeral jobs
created via /runs/submit
    print(
        "Scanning workspace for runs with Spark Submit tasks from the
last 30 days... (this will take more than an hour in large workspaces)",
        file=sys.stderr,
    )
    runs_with_spark_submit = []
    total_runs = 0
    seen_job_ids = set()

    # Iterate through all runs (pagination is handled automatically by
the SDK)
    skipped_runs = 0
    for run in client.jobs.list_runs(
        expand_tasks=True,
        limit=25,
        completed_only=True,
        start_time_from=thirty_days_ago_ms,
    ):
        try:
            total_runs += 1
            if total_runs % 1000 == 0:
                print(f"Scanned {total_runs} runs total",
file=sys.stderr)

            # Check if run has any Spark Submit tasks
            if run.tasks:
```

```
                has_spark_submit = any(
                    task.spark_submit_task is not None for task in
run.tasks
                )

                if has_spark_submit:
                    # Extract job information from the run
                    job_id = run.job_id if run.job_id else "N/A"
                    run_id = run.run_id if run.run_id else "N/A"
                    owner_email = run.creator_user_name or "Unknown"
                    # Use run name if available, otherwise try to
construct a name
                    run_name = run.run_name or (
                        f"Run {run_id}" if run_id != "N/A" else "Unnamed
Run"
                    )

                    # Track unique job IDs to avoid duplicates for
persistent jobs
                    # (ephemeral jobs may have the same job_id across
multiple runs)
                    key = (job_id, run_id)
                    if key not in seen_job_ids:
                        seen_job_ids.add(key)
                        runs_with_spark_submit.append(
                            {
                                "job_id": job_id,
                                "run_id": run_id,
                                "owner_email": owner_email,
                                "job_name": run_name,
                            }
                        )
            except PermissionDenied:
                # Skip runs that the user doesn't have permission to access
                skipped_runs += 1
                continue

    # Print summary to stderr
    print(f"Scanned {total_runs} runs total", file=sys.stderr)
    if skipped_runs > 0:
        print(
            f"Skipped {skipped_runs} runs due to insufficient
permissions",
            file=sys.stderr,
        )
    print(
        f"Found {len(runs_with_spark_submit)} runs with Spark Submit
tasks",
        file=sys.stderr,
    )
```

```
    print("", file=sys.stderr)

    # Output CSV to stdout
    if runs_with_spark_submit:
        writer = csv.DictWriter(
            sys.stdout,
            fieldnames=["job_id", "run_id", "owner_email", "job_name"],
            quoting=csv.QUOTE_MINIMAL,
        )
        writer.writeheader()
        writer.writerows(runs_with_spark_submit)
    else:
        print("No runs with Spark Submit tasks found.", file=sys.stderr)


if __name__ == "__main__":
    main()
```

Need help?

If you need additional help, please contact your account support.

*Last updated on **Jan 23, 2026***

Run Job task for jobs

Use the **Run Job** task to trigger other jobs configured in your Databricks workspace.

IMPORTANT

Databricks does not support jobs with circular dependencies or that nest more than three `Run Job` tasks. Databricks might block deploying jobs with this pattern in a future release.

Circular dependencies are `Run Job` tasks that directly or indirectly trigger each other. For example:

- Job A triggers Job B.
- Job B triggers Job A.

Configure a Run Job task

Add a `Run Job` task from the **Tasks** tab in the Jobs UI by doing the following:

1. Click **Add task**.
2. Enter a name into the **Task name** field.
3. In the **Type** drop-down menu, select `Run Job`.
4. In the **Job** drop-down menu, select a job to be run by the task. To search for the job to run, start typing the job name in the **Job** menu.
5. (Optional) Configure **Job parameters** to be passed to the specified job.

   Job parameters are pushed down. Similar to **Run now with different parameters**, you can override the default parameters for the specified job.

6. Click **Save task**.

Last updated on ***Apr 30, 2026***

Add branching logic to a job with the If/else task

Use the `If/else condition` task to add boolean conditional logic to task graphs. These tasks consist of a boolean operator and a pair of operands, where the operands might reference the job or task state using configured or dynamic parameters or task values. See Parameterize jobs.

For example, suppose you have a task named `process_records` that maintains a count of records that are not valid in a value named `bad_records`, and you want to branch processing when you encounter bad records. To add this logic to your workflow, you can create an `If/else condition` task with an expression like `{{tasks.process_records.values.bad_records}} > 0`. You can then add dependent tasks based on the results of the condition.

After a job run containing an `If/else condition` task, you can view the result and the expression evaluation details when you view the job run details in the UI. See View job run details.

NOTE

- Numeric and non-numeric values are handled differently depending on the boolean operator:
  - The `==` and `!=` operators perform string comparison of their operands. For example, `12.0 == 12` evaluates to false.
  - The `>`, `>=`, and `<=` operators perform numeric comparisons of their operands. For example, `12.0 >= 12` evaluates to true, and `10.0 >= 12` evaluates to false.
  - Only numeric, string, and boolean values are allowed when referencing task values in an operand. Any other types will cause the condition expression to fail. Non-numeric value types are serialized to strings and are treated as strings in `If/else condition` expressions. For

> a task value is set to a boolean value, it is serialized to `"true"` or `"false"`.

Configure an If/else task

Add an `If/else condition` task from the **Tasks** tab in the Jobs UI by doing the following:

1. Click **Add task**.
2. Enter a name into the **Task name** field.
3. In the **Type** drop-down menu, select `If/else condition`.
4. Enter the operand to be evaluated in the first **Condition** text box. The operand can reference any of the following:
   - A job parameter variable.
   - A task parameter variable.
   - A task value.
5. Select a boolean operator from the drop-down menu.
6. In the second **Condition** text box, enter the value for evaluating the condition.
7. Click **Save task**.

Configure dependencies on an If/else condition

Configure dependencies on the `If/else condition` task from the tasks graph in the **Tasks** tab by doing the following:

1. Select the `If/else condition` task in the tasks graph and click **+ Add task**.
2. Enter details for the task. The **Depends on** field defaults to `<task-name> (true)` where `<task-name>` is the name of the `If/else condition` task.
   - Select `<task-name> (false)` to configure a task that runs on a false condition evaluation.

You can configure multiple tasks to run in serial or parallel based on the outcome of an `If/else condition`. Consider configuring `Run if dependencies` if you need

conditionalized runs based on upstream task failures. See Configure task dependencies.

> **NOTE**
>
> An `If/else condition` task fails if the upstream task that provides its condition value is disabled. See Limitations.

Last updated on ***Apr 30, 2026***

Use a `For each` task to run another task in a loop

This article discusses using the `For each` task with your Lakeflow Jobs, including details on adding and configuring the task in the Jobs UI. Use the `For each` task to run a nested task in a loop, passing a different set of parameters to each iteration of the task.

Adding the `For each` task to a job requires defining two tasks: The `For each` task and a *nested task*. The nested task is the task to run for each iteration of the `For each` task and is one of the standard Lakeflow Jobs task types. You cannot add another `For each` task as the nested task.

For example, you could use the `For each` task to perform a common set of transformations on multiple tables, passing a table name from a list of table names to each iteration of the task.

Nested tasks that do not have dependencies on each other can be run concurrently.

Add the `For each` task to a job

You can add a `For each` task when you create a job or edit a task in an existing job. To configure a `For each` task:

1. Click **Add task**.
2. Enter a name into the **Task name** field.
3. In the **Type** drop-down menu, select **For each**.
4. Enter a name for the task in the **Task name** field.

5. In the **Inputs** text box, define the values for the `For each` task to iterate on as a JSON formatted array of values. To learn more about passing parameters to the nested task, see What parameter types can I use with the `For each` task?.

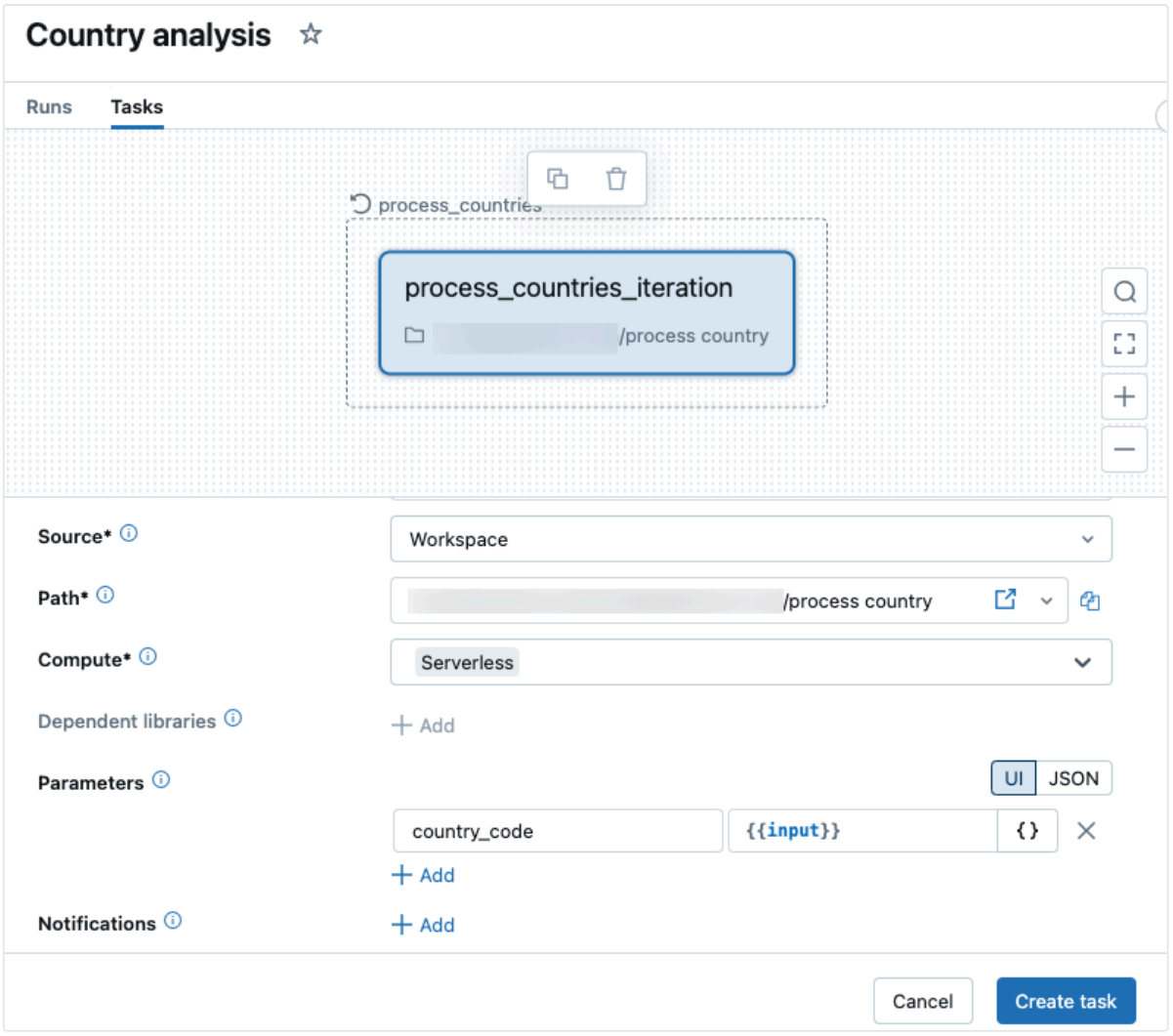
6. To optionally set the number of iterations that can run in parallel, enter a **Concurrency** value for the task. The default value is 1.

7. To optionally receive notifications for task start, success, or failure, click **+ Add**. See Add notifications on a job.

8. To complete the configuration of the `For each` task and add a nested task to run for each iteration, click **Add a task to loop over**.

9. Select a task type and configuration options for the nested task. Nested tasks are standard task types and have the same configuration options. See Configure and edit tasks in Lakeflow Jobs.

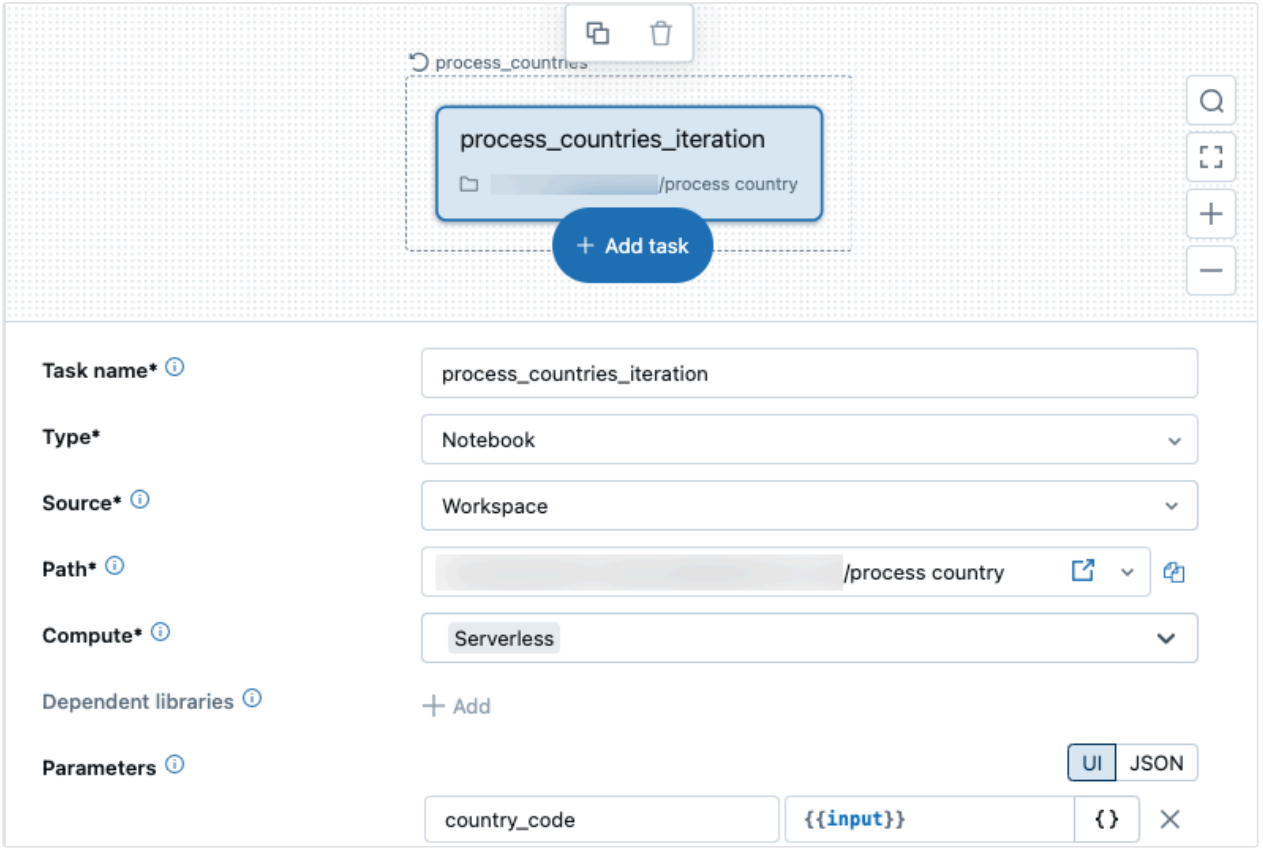
10. To reference parameters passed from the `For each` task, click **Parameters**. Use the `{{input}}` reference to set the value to the array value of each iteration or `{{input.<key>}}` to reference individual object fields when you iterate over a list of objects.



11. Click **Create task**.

Switch between the `For each` task and the nested task

The `For each` task appears in the Jobs UI as a node with the nested task node inside the `For each` node. To switch between the `For each` task and the nested task, click the respective nodes.



What parameter types can I use with the `For each` task?

The `For each` task passes parameters to each iteration of the nested task. The input is an array of objects, and each object is passed to an iteration of the nested task.

There are multiple ways to create the inputs that the task uses: JSON formatted arrays, task values, or job parameters.

> **NOTE**
>
> Parameters are limited to 5,000 characters in the UI directly, or 48 KB if you use task value references (where the value can be much larger than the size of the string that describes it), or 10,000 characters for the value of job parameters. If your parameters require more than 48 KB, you can pass a lookup to a larger config file. See Use a lookup table for large parameter arrays.

A JSON formatted array of values

When you create or edit a task, you can directly define an array of values for the nested task, using the **Inputs** text box. This can be an array of the following data types:

- Key-value pairs
- Strings, numbers, or Boolean types
- Arbitrarily complex JSON objects

The **Inputs** text box, and therefore JSON passed directly in this box, is limited to 5,000 characters.

Task value references

You can pass task values from a preceding task. To reference passed task values, use the `{{tasks.<task_name>.values.<task_value_name>}}` syntax to set the value in the **Inputs** text box. For example, if a task named `generate_countries_list` that precedes the `For each` task sets the following task value:

```
dbutils.jobs.taskValues.set(key = "countries", value = countries_array)
```

Then the `For each` task references the task value in the **Inputs** text box using the following syntax:

`{{tasks.generate_countries_list.values.countries}}`.

You can put up to 5,000 characters in the **Inputs** text box, but the values that the references represent are able to be up to 48 KB. To learn more about task values, see Use task values to pass information between tasks.

Job parameters

You can also use job parameters as input. To reference a job parameter, use the following syntax in the **Inputs** text box: `{{job.parameters.<name>}}`. For example, `{{job.parameters.countries}}`.

You can put up to 5,000 characters in the **Inputs** text box to refernce job parameters. The job parameter values are limited to 10 KB. To learn more about job parameters, see Configure job parameters.

Reference a `For each` task in downstream tasks

The `For each` task is the top-level task, and downstream tasks can specify it as a dependency. Downstream tasks cannot depend on or reference the nested task.

> **NOTE**
>
> A `For each` task fails if the upstream task that provides its input values is disabled. See Limitations.

Run and monitor a job with a `For each` task

Running a job with a `For each` task is identical to running any other job.

Viewing and managing job runs is also identical to any other job, except the task run history for a `For each` task, which is presented as a table of task iterations. See View task run history for a `For each` task.

Tutorial

To build a metadata-driven job using a SQL task and a `For each` task, see Use a control table to drive a `For each` job.

Last updated on ***Apr 30, 2026***

Configure task dependencies

The **Run if dependencies** field allows you to add control flow logic to tasks based on other tasks' success, failure, or completion.

Dependencies are visually represented in the job DAG as lines between tasks.

Databricks runs upstream tasks before running downstream tasks, running as many of them in parallel as possible.

> **NOTE**
>
> **Depends on** is only visible if the job consists of multiple tasks.

Databricks also has the following functionality for control flow and conditionalization:

- The *If/else condition* task is used to run a part of a job DAG based on the results of a boolean expression. The `If/else condition` task allows you to add branching logic to your job. For example, run transformation tasks only if the upstream ingestion task adds new data. See Add branching logic to a job with the If/else task.
- The *For each* condition task adds looping logic to another task based on an input array. Input arrays can be manually specified or dynamically generated. See Use a `For each` task to run another task in a loop.
- The *Run Job* task lets you trigger another job in your workspace. See Run Job task for jobs.

Add a Run if condition to a task

If you have a task selected in your DAG when you create a new task, your new task has a dependency configured on this task by default.

To edit or add conditions, do the following:

1. Select a task.
2. In the **Depends on** field, click the **X** to remove a task or select tasks to add from the drop-down menu.
3. Select one of the conditional options in the **Run if dependencies** field.
4. Click **Save task**.

`Run if` condition options

You can add the following `Run if` conditions to a task:

- **All succeeded**: All dependencies have run and succeeded. This is the default setting. The task is marked as `Upstream failed` if the condition is unmet.
- **At least one succeeded**: At least one dependency has succeeded. The task is marked as `Upstream failed` if the condition is unmet.
- **None failed**: None of the dependencies failed, and at least one dependency was run. The task is marked as `Upstream failed` if the condition is unmet.
- **All done**: The task is run after all its dependencies have run, regardless of the status of the dependent runs. This condition allows you to define a task that is run without depending on the outcome of its dependent tasks.
- **At least one failed**: At least one dependency failed. The task is marked as `Excluded` if the condition is unmet.
- **All failed**: All dependencies have failed. The task is marked as `Excluded` if the condition is unmet.

NOTE

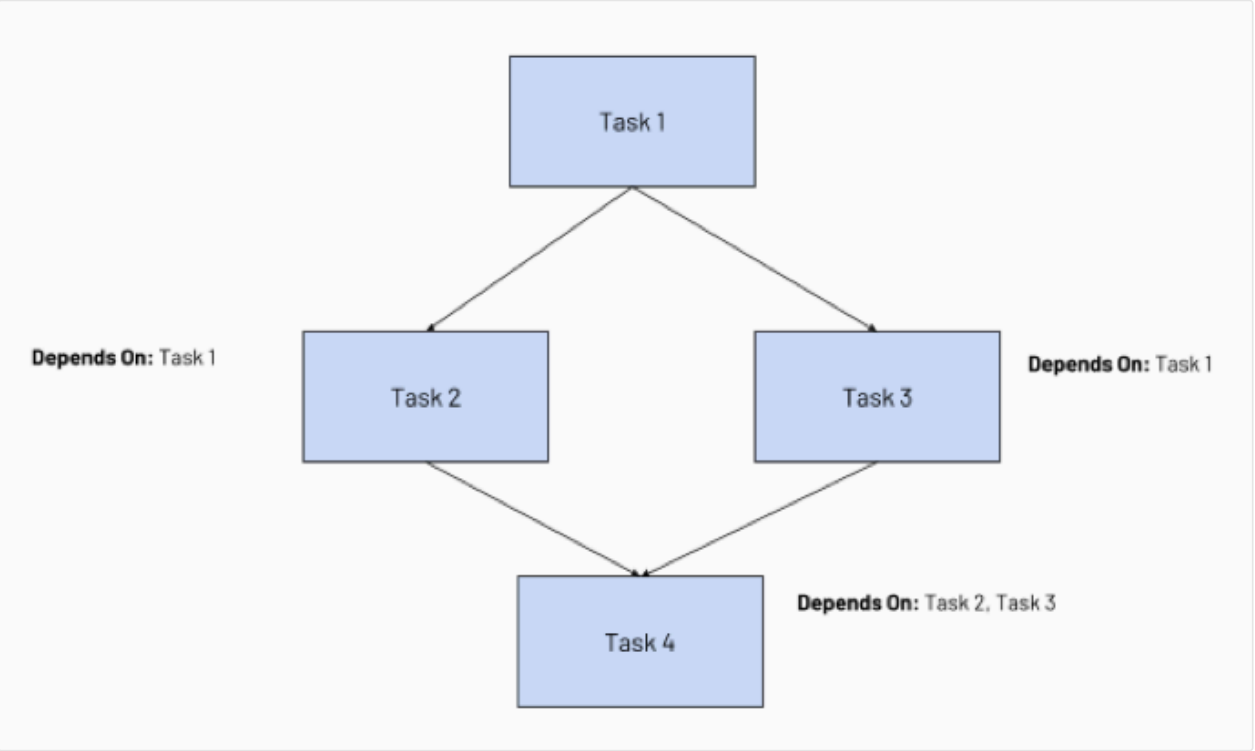
- Skipped (`Excluded`) upstream tasks are treated as successful when evaluating `Run if` conditions.
- Tasks configured to handle failures are marked as `Excluded` if their `Run if` condition is unmet. Excluded tasks are skipped and are treated as successful.
- If all task dependencies are excluded, the task is also excluded, regardless of its `Run if` condition.
- If you cancel a task run, the cancellation propagates through downstream tasks, and tasks with a `Run if` condition that handles failure are run, for

example, to verify a cleanup task runs when a task run is canceled.

- If an upstream task is disabled, downstream tasks are evaluated against their `Run if` condition and may also be disabled for that run. For the full list of outcomes, see Downstream task behavior.

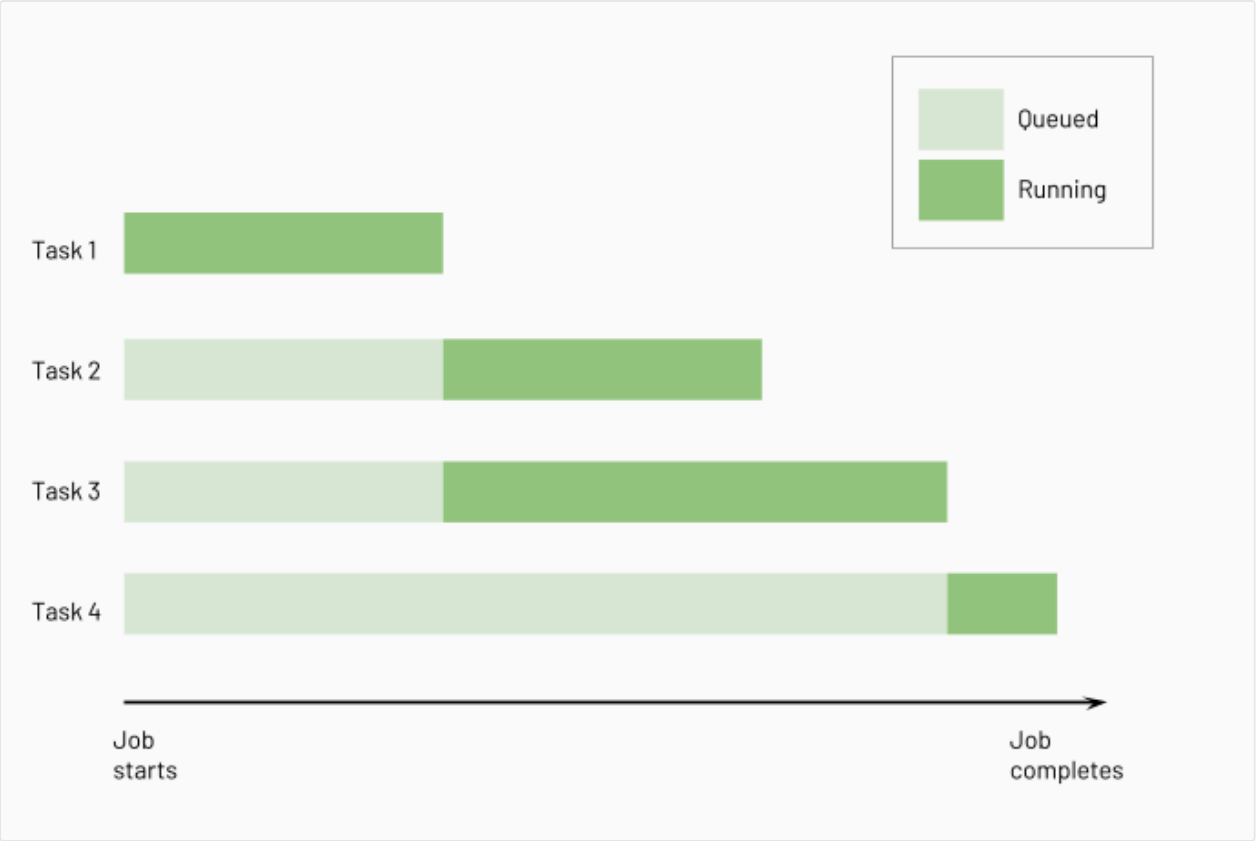
Example job with task dependencies

Configuring task dependencies creates a Directed Acyclic Graph (DAG) of task execution, a common way of representing execution order in job schedulers. For example, consider the following job consisting of four tasks:



- Task 1 is the root task and does not depend on any other task.
- Task 2 and Task 3 depend on Task 1 completing first.
- Finally, Task 4 depends on Task 2 and Task 3 completing successfully.

The following diagram illustrates the order of processing for these tasks:



this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

Last updated on ***Apr 30, 2026***

Disabled tasks in Lakeflow Jobs

A disabled task in a Databricks Lakeflow job is skipped at run time without being removed from the job. Disabled tasks keep their configuration and run history, so you can re-enable them later without rebuilding the task. This page explains how disabled tasks behave when a job runs, including their effect on downstream tasks, repairs, and partial runs.

Downstream task behavior

When a job runs, Lakeflow Jobs evaluates each downstream task's **Run if** condition against its upstream tasks to decide whether to run, skip, or disable the task. Disabled tasks complete with a `Disabled` termination code.

If a downstream task's **Run if** condition can't be met because one or more parent tasks are disabled, Lakeflow Jobs also marks the downstream task as disabled for that run. Disabled downstream tasks show a ∅ in the upper-right corner of the Directed Acyclic Graph (DAG) view, so you can see the impact before starting a run.

The following table summarizes downstream behavior for each **Run if** condition when an upstream task is disabled. For the full list of **Run if** options, see Configure task dependencies.

Run if condition	**Downstream task behavior when a parent task is disabled**	**Example**
All succeeded (default)	The downstream task does not run. A disabled parent task does not satisfy the `succeeded` requirement.	`A (disabled) → B`: `B` does not run.

Run if condition	Downstream task behavior when a parent task is disabled	Example
At least one succeeded	The downstream task runs if at least one other parent task succeeded. If all other parent tasks failed or were disabled, the downstream task does not run.	`A (disabled)` and `C (succeeded) → B`: `B` runs.
None failed	The downstream task runs if at least one parent task completed without failure. If all parent tasks are disabled, the downstream task does not run.	`A (disabled)` and `C (skipped) → B`: `B` runs because no parent task failed.
All done	The downstream task runs normally. A disabled parent task is treated as done.	`A (disabled) → B`: `B` runs.
At least one failed	The downstream task runs if at least one other parent task failed. A disabled parent task is not treated as a failure. If no other parent task failed, the downstream task does not run.	`A (disabled)` and `C (failed) → B`: `B` runs.
All failed	The downstream task does not run. A disabled parent task is not treated as a failure.	`A (disabled) → B`: `B` does not run.

NOTE

Only tasks that you explicitly disable have `disabled: true` in the job definition. Lakeflow Jobs determines downstream disablement at run creation time and doesn't persist it in the job settings.

Disable a task

To disable or re-enable a task using the UI, see Disable a task.

To disable a task through the API or a bundle:

Set `disabled: true` on the task in the job settings using the Jobs REST API, the Databricks CLI, the Databricks SDK, or a Declarative Automation Bundles:

JSON

```
{
  "tasks": [
    {
      "task_key": "load_raw_data",
      "disabled": true,
      "notebook_task": {
        "notebook_path": "/Shared/etl/load_raw_data"
      }
    }
  ]
}
```

The `jobs/get` and `jobs/list` responses return `disabled: true` only for tasks that you explicitly disabled. Tasks disabled dynamically during a run aren't reflected in the stored job settings.

Disabled tasks in repairs and partial runs

Disabled tasks behave differently in repair runs and partial runs than in a typical scheduled run:

- **Repairs:** Lakeflow Jobs uses the run state of each task to determine what to repair, not the task's disabled state. To force a disabled task to run as part of a repair, include it in `rerun_tasks` in the repair request. See Re-run failed and skipped tasks.
- **Partial runs:** Disabled tasks aren't selected by default when you start a partial run, but you can select them to run once without re-enabling them in the job settings. Lakeflow Jobs runs exactly the tasks you select and doesn't apply **Run if** propagation during a partial run.

Limitations

Disabled tasks have the following limitations:

- An `If/else condition` task fails if the upstream task that provides its condition value is disabled.
- A `For each` task fails if the upstream task that provides its input values is disabled.
- Only user-disabled tasks appear as `disabled: true` in the job definition. To see which downstream tasks are affected before running the job, use the DAG view in the Jobs UI.

*Last updated on **Jan 23, 2026***

Parameterize jobs

This article provides an overview of using parameters with jobs and tasks, focusing on the specifics of configuration for jobs and tasks in the UI. You must also update the source code assets you configure as tasks to reference parameters. Parameter references vary by language and task type. See Access parameter values from a task.

The following are foundational concepts for understanding parameters for jobs:

- **Job parameter**: A key-value pair defined at the job level and pushed down to tasks. See Configure job parameters.
- **Task parameter**: A key-value pair or JSON array defined at the task level. See Configure task parameters.
- **Dynamic value references**: A syntax for references job conditions, metadata, and parameters when configuring tasks. See What is a dynamic value reference?.
- **Task values**: A syntax for capturing and referencing values generated during task runs. See Use task values to pass information between tasks.

What can you do with parameters?

Add parameters to jobs and tasks for advanced use cases including the following:

- Add extensible logic to code assets.
- Conditionalize runs.
- Reference common parameters across multiple tasks.
- Use information generated in one task in another task.
- Reference metadata and state information in the job run.

What is the difference between job and task parameters?

Job parameters are key-value pairs defined at the job level. You can override the default settings for job parameters when you **Run now with different parameters** or trigger a job run using the REST API. Job parameters are pushed down to tasks that use key-value parameters. Other tasks can reference job parameters using dynamic value references.

Task parameters are key-value pairs of JSON arrays defined at the task level. Each task type passes task values to the configured code assets differently. For example, notebook tasks use the `dbutils.widgets` submodule, while Python scripts pass values as arguments to the script as if it were being called from the command line. Downstream tasks can reference task parameters from upstream tasks using dynamic value references. See Access parameter values from a task.

> **NOTE**
>
> Some tasks do not have a dedicated **Parameters** field, but allow references to task values or dynamic value references within other fields. See Examples of parameterized dbt commands and Add branching logic to a job with the If/else task.

Build workflows using dynamic values

Task parameters set with static values can only be overridden by updating the task definition. Setting a static value for a job parameter is just configuring a default value, which you can override when you **Run now with different parameters** or trigger a job run using the REST API.

Use dynamic value references when defining task parameters to implement patterns such as the following:

- Use a job parameter as the `output_table` for one task and the `input_table` for another.
- Capture the output of a notebook query as a list and loop over it in a **For each** task.
- Creating forking logic based on the number of records processed using an **If/else condition** task.
- Refer to parameters from other tasks.

See What is a dynamic value reference?.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **Jan 23, 2026***

Configure job parameters

This article describes job parameter functionality and configuring job parameters with the Databricks workspace UI. You can also add job parameters to JSON and YAML definitions used with the REST API, CLI, and Declarative Automation Bundles. See Jobs API, Databricks CLI, and What are Declarative Automation Bundles?.

What are job parameters?

Job parameters are key-value pairs that allow you to parameterize jobs with default static or dynamic values. You can optionally override parameters configured in a job when triggering a new run. See Run a job with different parameters.

Job parameter keys can only contain `_ - .` or alphanumeric characters. Parameter values are set as strings or dynamic value references. See What is a dynamic value reference?.

> **NOTE**
>
> You can use any valid JSON as a parameter value. For example, the `For each` task type can parse lists such as the following:
>
> ```
> [1, 2, 3]
> ['a', 'b', 'c']
> ```

Add or edit job parameters

Use the **Job parameters** dialog to add new parameters, edit existing parameter keys and values, or delete parameters.

To edit parameters with the workspace UI, select an existing job using the following steps:

1. In your Databricks workspace's sidebar, click **Jobs & Pipelines**.
2. Optionally, select the **Jobs** and **Owned by me** filters.
3. Click your job's **Name** link.
4. In the **Job details** sidebar, click **Edit parameters**. The **Job parameters** dialog appears.
5. Add or edit parameters using **Key** and **Value** fields.
6. Click the  to remove a parameter.
7. Click **Save** to apply your changes.

> **NOTE**
>
> Click **{ }** to list available dynamic value references. Select an option from the list to insert it into the **Value** field.

Job parameter pushdown

Job parameters are pushed to different task types in different ways. To understand how they are passed, and how to access them within the task, see Access parameter values from a task.

> **IMPORTANT**
>
> Job parameters take precedence over task parameters. If a job parameter and a task parameter have the same key, the job parameter overrides the task parameter.

Run a job with different parameters

You can override configured job parameters or add new ones when you run a job with different parameters. See Run a job with different settings.

You can also override job parameters when you repair a job run. See Re-run failed and skipped tasks.

Last updated on ***Apr 9, 2026***

Configure task parameters

Task parameters allow you to parameterize tasks using values that can be static, dynamic, or set by upstream tasks.

For information on using dynamic values, see What is a dynamic value reference?.

For information on passing context between tasks, see Use task values to pass information between tasks.

The assets configured by tasks use different syntax to refer to values passed as parameters. See Access parameter values from a task.

> **NOTE**
>
> Some tasks support parameterization but do not have parameter fields. See the following:
>
> - Examples of parameterized dbt commands
> - Add branching logic to a job with the If/else task

Configure key-value parameters

Configure parameters for the following tasks as key-value pairs:

- Notebook
- Python wheel (only when configured with keyword arguments)
- SQL query or file
- Run Job

Job parameters are automatically pushed down to tasks that support key-value parameters. A warning is shown in the UI if you attempt to add a task parameter with the same key as a job parameter. See Job parameter pushdown.

Configure JSON array parameters

Configure parameters for the following tasks as a JSON-formatted array of strings:

- Python script
- Python wheel (only when configured with positional arguments)
- JAR
- Spark Submit
- For each

The **For each** task iterates over this array to run conditionalized logic on the configured task.

All other task types pass the contents of the JSON-formatted array as arguments as if the configured code assets were being run from the command line.

Job parameters are not automatically pushed down to tasks that use JSON arrays. You can reference job parameters in the task JSON-formatted array using the dynamic value reference `{{job.parameters.<name>}}`.

> **NOTE**
>
> Job parameter values can include any valid JSON construct. This means that you can use dynamic value references to job parameters to conditionalize tasks.

For details about how to access parameters in your task code, see Access parameter values from a task.

Last updated on ***Apr 14, 2026***

What is a dynamic value reference?

Dynamic value references describe a collection of variables available when configuring jobs and tasks. Use dynamic value references to configure conditional statements for tasks or to pass information as parameters or arguments.

Dynamic value references include information such as:

- Configured values for the job, including the job name, task names, and trigger type.
- Generated metadata about the job, including the job ID, the run ID, and the job run start time.
- Information about how many repair attempts a job has made or retries a task has run.
- The result state for a specified task.
- Values configured using job parameters, task parameters, or set using task values.

Use dynamic value references

Use dynamic value references when configuring jobs or tasks. You cannot directly reference dynamic value references from assets configured using tasks like notebooks, queries, or JARs. Dynamic value references must be defined using parameters or fields that pass context into tasks.

Dynamic value references use double curly braces (`{{ }}`). When a job or task runs, a string literal replaces the dynamic value reference. For example, if you configure the following key-value pair as a task parameter:


```
{"job_run_id": "job_{{job.run_id}}"}
```

If your run ID is `550315892394120`, the value for `job_run_id` evaluates to `job_550315892394120`.

> **NOTE**
>
> Contents of the double curly braces are not evaluated as expressions. You cannot run operations or functions in double-curly braces.
>
> User-provided value identifiers support alphanumeric and underscore characters. Escape keys that contain special characters by surrounding the identifier with backticks (`` ` ` ``).
>
> Syntax errors, including non-existent dynamic reference values and missing braces, are silently ignored and are treated as literal strings. An error message is displayed if you provide an not valid reference belonging to a known namespace, for example, `{{job.notebook_url}}`.

Use dynamic value references in the jobs UI

Fields that accept dynamic value references provide a shortcut to insert available dynamic value references. Click **{ }** to see this list and insert it into the provided field.

> **NOTE**
>
> The UI does not auto-complete the keys to reference task values.
>
> Many fields that accept dynamic value references require additional formatting to use them correctly. See Configure task parameters.

Use dynamic value references in a job JSON

Use `{{ }}` syntax to use dynamic values in job JSON definitions used by the Databricks CLI and REST API.

Job and task parameters have different syntax, and task parameter syntax varies by task type.

The following example shows the partial JSON syntax to configure job parameters using dynamic value references:

JSON

```
{
  "parameters": [
    {
      "name": "my_job_id",
      "default": "{{job.id}}"
    },
    {
      "name": "run_date",
      "default": "{{job.start_time.iso_date}}"
    }
  ]
}
```

The following example shows the partial JSON syntax to configure notebook task parameters using a dynamic value reference:

JSON

```
{
  "notebook_task": {
    "base_parameters": {
      "workspace_id": "workspace_{{workspace.id}}",
      "file_arrival_location": "{{job.trigger.file_arrival.location}}"
    }
  }
}
```

Review parameters for a job run

After a task completes, you can see resolved parameter values under **Parameters** on the run details page. See View job run details.

Supported value references

The following dynamic value references are supported:

Reference
`{{job.id}}`
`{{job.name}}`
`{{job.run_id}}`
`{{job.repair_count}}`
`{{job.start_time.<argument>}}`
`{{job.parameters.<name>}}`
`{{job.trigger.type}}`
`{{job.trigger.file_arrival.location}}`
`{{job.trigger.table_update.updated_tables}}`
`{{job.trigger.table_update.`<catalog.schema.table>`.commit_timestamp.iso_dat`
`{{job.trigger.table_update.`<catalog.schema.table>`.version}}`

Reference
{{job.trigger.time.<argument>}}
{{task.name}}
{{task.run_id}}
{{task.execution_count}}
{{task.notebook_path}}
{{tasks.<task_name>.run_id}}
{{tasks.<task_name>.output.catalog_name}}
{{tasks.<task_name>.output.schema_name}}
{{tasks.<task_name>.result_state}}
{{tasks.<task_name>.error_code}}
{{tasks.<task_name>.execution_count}}
{{tasks.<task_name>.notebook_path}}

Reference
`{{tasks.<task_name>.values.<value_name>}}`
`{{tasks.<task_name>.output.rows}}`
`{{tasks.<task_name>.output.first_row}}`
`{{tasks.<task_name>.output.first_row.<column_alias>}}`
`{{tasks.<task_name>.output.alert_state}}`
`{{workspace.id}}`
`{{workspace.url}}`
`{{backfill.day}}`
`{{backfill.is_weekday}}`
`{{backfill.iso_date}}`
`{{backfill.iso_datetime}}`

Reference
`{{backfill.iso_weekday}}`
`{{backfill.month}}`
`{{backfill.timestamp_ms}}`
`{{backfill.year}}`

You can set these references with any task. See Configure task parameters. Backfill parameters are only available when creating a backfill job.

You can also pass parameters between tasks in a job with *task values*. See Use task values to pass information between tasks.

Options for date and time values

Use the following arguments to specify the return value from time-based parameter variables. All return values are based on a timestamp in the UTC timezone.

Argument	Description
`iso_weekday`	Returns a digit from 1 to 7, representing the day of the week of the timestamp.
`is_weekday`	Returns `true` if the timestamp is on a weekday.
`iso_date`	Returns the date in ISO format.
`iso_datetime`	Returns the date and time in ISO format.

Argument	Description
`year`	Returns the year part of the timestamp.
`month`	Returns the month part of the timestamp.
`day`	Returns the day part of the timestamp.
`hour`	Returns the hour part of the timestamp.
`minute`	Returns the minute part of the timestamp.
`second`	Returns the second part of the timestamp.
`timestamp_ms`	Returns the timestamp in milliseconds.

SQL output options

You can access the output of an upstream SQL task using dynamic values.

For example, if you have a SQL task called `sales_by_year` with the following SQL, it creates output based on the final `SELECT` statement.

SQL

```sql
-- Generate example data
CREATE OR REPLACE TEMP VIEW example_sales AS
SELECT * FROM VALUES
  (2020, 12),
  (2021, 23),
  (2022, 47),
  (2023, 15),
  (2024, 22)
AS example_sales(sales_year, num_sales);

-- Query example data
SELECT sales_year, num_sales FROM example_sales;
```

You can reference the output in a downstream task by creating task configuration using a `{{tasks.<task_name>.output.<argument>}}` dynamic value.

In `For each` tasks, the rows are sent iteratively to the nested task. In this example, if you were to set the **Inputs** of a `For each` task to `{{tasks.sales_by_year.output.rows}}`, then in the nested task, you can use the syntax `{{input.<column_alias>}}` to iteratively send the row values as parameters. In the nested task configuration, you could create two parameters with the key-value pairs `year`:`{{input.sales_year}}`, and `sales`:`{{input.num_sales}}`. Assuming the nested task is a SQL task, you could reference the values in your code with a query such as the following.

SQL

```
-- Example: access data from previous query:
SELECT concat('In ', :year, ' we had ', :sales, ' sales.')
```

For more information about `For each` tasks and their nested tasks, see Use a `For each` task to run another task in a loop.

> **NOTE**
>
> The SQL query output is retained for 7 days. If the job is paused (for example, on failure), then resumed more than 7 days later, the SQL query output will not be available.

Deprecated dynamic value references

The following dynamic value references are deprecated. The recommended replacement reference is included in the description of each variable.

Variable	Description
`{{job_id}}`	The unique identifier assigned to a job. Use `job.id` instead.

Variable	Description
`{{run_id}}`	The unique identifier assigned to a task run. Use `task.run_id` instead.
`{{start_date}}`	The date a task run started. The format is YYYY-MM-DD in the UTC timezone. Use `job.start_time.<argument>` instead.
`{{start_time}}`	The timestamp of the run's start of execution after the cluster is created and ready. The format is milliseconds since UNIX epoch in the UTC timezone, as returned by `System.currentTimeMillis()`. Use `job.start_time.<format>` instead.
`{{task_retry_count}}`	The number of retries attempted to run a task if the first attempt fails. The value is 0 for the first attempt and increments with each retry. Use `task.execution_count` instead.
`{{parent_run_id}}`	The unique identifier assigned to the run of a job with multiple tasks. Use `job.run_id` instead.
`{{task_key}}`	The unique name assigned to a task that's part of a job with multiple tasks. Use `task.name` instead.

*Last updated on **Jan 23, 2026***

Use task values to pass information between tasks

Task values refer to the Databricks Utilities `taskValues` subutility, which lets you pass arbitrary values between tasks in a Databricks job. See taskValues subutility (dbutils.jobs.taskValues).

You specify a key-value pair using `dbutils.jobs.taskValues.set()` in one task and then can use the task name and key to reference the value in subsequent tasks.

> **NOTE**
>
> Because `dbutils.jobs.taskValues.set()` and `dbutils.jobs.taskValues.get()` in the `dbutils.jobs.taskValues` subutility are Python functions, they can be used only in notebooks with Python selected as the language. However, you can reference task values using dynamic value references for all tasks that support parameters. See Reference task values.

Set task values

Set task values in Python notebooks using `dbutils.jobs.taskValues.set()`.

Task value keys must be strings. Each key must be unique if you have multiple task values defined in a notebook.

You can manually or programmatically assign task values to keys. Only values that can be expressed as valid JSON are permitted. The size of the JSON representation of the value cannot exceed 48 KiB.

For example, the following example sets a static string for the key `fave_food`:

Python

```
dbutils.jobs.taskValues.set(key = "fave_food", value = "beans")
```

The following example uses a notebook task parameter to query all records for a particular order number and return the current order status and total count of records:

Python

```
from pyspark.sql.functions import col

order_num = dbutils.widgets.get("order_num")

query = (spark.read.table("orders")
  .orderBy(col("updated"), ascending=False)
  .select(col("order_status"))
  .where(col("order_num") == order_num)
)

dbutils.jobs.taskValues.set(key = "record_count", value = query.count())
dbutils.jobs.taskValues.set(key = "order_status", value = query.take(1)
[0][0])
```

You can pass lists of values using this pattern and then use them to coordinate downstream logic, such as for each tasks. See Use a `For each` task to run another task in a loop.

The following example extracts the distinct values for product ID to a Python list and sets this as a task value:

Python

```
prod_list =
list(spark.read.table("products").select("prod_id").distinct().toPandas()
["prod_id"])

dbutils.jobs.taskValues.set(key = "prod_list", value = prod_list)
```

Reference task values

Databricks recommends referencing task values as task parameters configured using the dynamic value reference pattern `{{tasks.<task_name>.values.<value_name>}}`.

For example, to reference the task value with the key `prod_list` from a task named `product_inventory`, use the syntax `{{tasks.product_inventory.values.prod_list}}`.

See Configure task parameters and What is a dynamic value reference?

Use `dbutils.jobs.taskValues.get`

The syntax `dbutils.jobs.taskValues.get()` requires specifying the upstream task name. This syntax is not recommended, as you can use task values in multiple downstream tasks, meaning numerous updates are necessary if a task name changes.

Using this syntax, you can optionally specify a `default` value and a `debugValue`. The default value is used if the key cannot be found. The `debugValue` allows you to set a static value to use during manual code development and testing in notebooks before you schedule the notebook as a task.

The following example gets the value for the key `order_status` set in a task name `order_lookup`. The value `Delivered` is returned only when running the notebook interactively.

Python

```
order_status = dbutils.jobs.taskValues.get(taskKey = "order_lookup", key
= "order_status", debugValue = "Delivered")
```

NOTE

Databricks does not recommend setting default values, as they can be challenging to troubleshoot and prevent expected error messages due to missing keys or incorrectly named tasks.

View task values

The returned value of a task value for each run is displayed in the **Output** panel of the **Task run details**. See View task run history.

Last updated on ***Apr 16, 2026***

Access parameter values from a task

This article describes how to access parameter values from code in your tasks, including Databricks notebooks, Python scripts, and SQL files.

Parameters include user-defined parameters, values output from upstream tasks, and metadata values generated by the job. See Parameterize jobs.

While the details vary by task type, there are four common methods used to reference parameter values from source code:

- Databricks utilities widgets
- SQL named parameter syntax
- Dynamic values references
- Code arguments

In each of these cases, you reference the parameter's key to access its value. The key is sometimes referred to as the name of the parameter.

To see these techniques applied in a metadata-driven job that uses a SQL task and a `For each` task, see Use a control table to drive a `For each` job.

Use `dbutils` in code in a notebook

Notebook code running in a task can access the values of parameters with the dbutils library. The following example shows how to use `dbutils` in Python to get the value of a `year_param` task parameter that's passed into the notebook task.

Python

```
# Retrieve a job-level parameter
year_value = dbutils.widgets.get("year_param")

# Use the value in your code
display(babynames.filter(babynames.Year == year_value))
```

Parameters are accessed by name. If you have task parameters and job parameters that have the same name, the job parameters will be fetched.

The above code produces an error when run in a standalone notebook and not as part of a job because parameters are not sent into a standalone notebook. You can set a default for the `year_param` parameter with the following code:

Python

```
# Set a default (for when not running in a job)
dbutils.widgets.text("year_param", "2012", "Year Parameter")

# Retrieve a job-level parameter (will use default if it doesn't exist)
year_value = dbutils.widgets.get("year_param")

# Use the value in your code
display(babynames.filter(babynames.Year == year_value))
```

While this is helpful for testing outside of a job, it also has the drawback of hiding when task or job parameters are not set up correctly.

Use named parameters in a SQL notebook

When you are running SQL in a notebook task, you can use the named parameter syntax to access task parameters. For example, to access a task parameter called `year_param`, you could get its value by using `:year_param` in your query:

SQL

```
SELECT *
FROM baby_names_prepared
```

Ask Genie

```
WHERE Year_Of_Birth = :year_param
GROUP BY First_Name
```

Access as code arguments

For some task types parameters are passed to the code as arguments. The following task types have arguments passed to them:

- `Python script`
- `Python Wheel`
- `JAR`
- `Spark Submit`

For details, see Details by task type, later in this article.

For `dbt` tasks, the parameters are passed by calling dbt commands in your task.

Use dynamic value references when configuring a task

When you are configuring a task in the Databricks UI, use the dynamic value reference syntax to access job parameters or other dynamic values. To access job parameters, use the syntax: `{{job.parameters.<name>}}`. For example, when configuring a `Python wheel` task, you can set a parameter's **Key** and **Value** inputs to reference a Job parameter called `year_param`, such as `year` / `Year_{{job.parameters.year_param}}`. Besides providing access to parameters in configuration, dynamic values also give you access to other data about your job or task, for example, the `{{job.id}}`. You can click **{}** in task configuration to get a list of possible dynamic values, and insert them into your configuration.

Details by task type

Which of these methods that you use depends on the task type.

Task type	Access in configuration	Access in code
Notebooks	You can use dynamic value references in the Databricks UI to configure the notebook (for example, to refer to job parameters in the task parameter values). You can override or add additional parameters when you manually run a task using the Run a job with different settings option.	You can use either named parameters for SQL in your notebook, or `dbutils.widgets` in your code.
Python script	Parameters defined in the task are passed as arguments to your script. You can use dynamic value references in the **Parameters** text box.	The parameters can be read as positional arguments or parsed using the argparse module in Python.
Python wheel	Parameters defined in the task definition are passed as keyword arguments to your code. Your Python wheel files must be configured to accept keyword arguments. You can use dynamic value references in the values of your parameters.	Access as keyword arguments to your script. To see an example of reading arguments in a Python script packaged in a Python wheel file, see Use a Python wheel file in Lakeflow Jobs.
SQL	You can use dynamic value references in your task configuration.	Use named parameters to access parameter values.
Pipeline	Pipelines do not support passing parameters to the task.	Not supported.
Dashboard	Dashboard tasks do not support passing parameters to the task.	Not supported.

Task type	Access in configuration	Access in code
Power BI	Power BI tasks do not support passing parameters to the task.	Not supported.
dbt	You can use dynamic value references to pass parameters as `dbt` commands when configuring your task.	Access as `dbt` commands.
JAR	You can use dynamic value references to pass parameters as arguments in the **Parameters** text box when configuring your task.	Parameters are accessed as arguments to the main method of the main class.
Spark Submit	You can use dynamic value references to pass parameters as arguments in the **Parameters** text box when configuring your task.	Parameters are accessed as arguments to main method of the main class.
Run Job	You can use dynamic value references to create a set of **Job Parameters** when configuring your task. The values can include dynamic value references.	Not applicable.
If/else condition	You can use dynamic value references when configuring your task, for example, in the **Condition**.	Not applicable.
For each	You can use dynamic value references when configuring the **Inputs** for your task. The nested task receives one input as a task parameter for each iteration of the nested task.	The nested tasks access parameters, based on the task type.

Task type	Access in configuration	Access in code
Clean room notebook	You can use dynamic value references in the Databricks UI to configure the notebook (for example, to refer to job parameters in the task parameter values).	You can use either named parameters for SQL in your notebook, or `dbutils.widgets` in your code.

*Last updated on **Mar 16, 2026***

Automate job creation and management

This article shows you how to get started with developer tools to automate the creation and management of jobs. It introduces you to the Databricks CLI, the Databricks SDKs, and the REST API.

> **NOTE**
>
> This article provides examples for creating and managing jobs using the Databricks CLI, the Databricks Python SDK, and the REST API as an easy introduction to those tools. To programmatically manage jobs as part of CI/CD use Declarative Automation Bundles or the Databricks Terraform provider.

Compare tools

The following table compares the Databricks CLI, the Databricks SDKs, and the REST API for programmatically creating and managing jobs. To learn about all available developer tools, see Local development tools.

Tool	Description
Databricks CLI	Access Databricks functionality using the Databricks command-line interface (CLI), which wraps the REST API. Use the CLI for one-off tasks such as experimentation, shell scripting, and invoking the REST API directly
Databricks SDKs	Develop applications and create custom Databricks workflows using a Databricks SDK, available for Python, Java, Go, or R. Instead

Tool	Description
	of sending REST API calls directly using curl or Postman, you can use an SDK to interact with Databricks.
Databricks REST API	If none of the above options work for your specific use case, you can use the Databricks REST API directly. Use the REST API directly for use cases such as automating processes where an SDK in your preferred programming language is not currently available.

Get started with the Databricks CLI

To install and configure authentication for the Databricks CLI, see Install or update the Databricks CLI and Authentication for the Databricks CLI.

The Databricks CLI has command groups for Databricks features, including one for jobs, that contain a set of related commands, which can also contain subcommands. The `jobs` command group enables you to manage your jobs and job runs with actions such as `create`, `delete` and `get`. Because the CLI wraps the Databricks REST API, most CLI commands map to a REST API request. For example, `databricks jobs get` maps to `GET/api/2.2/jobs/get`.

To output more detailed usage and syntax information for the jobs command group, an individual command, or subcommand, use the `h` flag:

- `databricks jobs -h`
- `databricks jobs <command-name> -h`
- `databricks jobs <command-name> <subcommand-name> -h`

Example: Retrieve a job using the CLI

To print information about an individual job in a workspace, run the following command:

Bash

Ask Genie

```
$ databricks jobs get <job-id>

databricks jobs get 478701692316314
```

This command returns JSON:

JSON


```
{
  "created_time": 1730983530082,
  "creator_user_name": "someone@example.com",
  "job_id": 478701692316314,
  "run_as_user_name": "someone@example.com",
  "settings": {
    "email_notifications": {
      "no_alert_for_skipped_runs": false
    },
    "format": "MULTI_TASK",
    "max_concurrent_runs": 1,
    "name": "job_name",
    "tasks": [
      {
        "email_notifications": {},
        "notebook_task": {
          "notebook_path":
"/Workspace/Users/someone@example.com/directory",
          "source": "WORKSPACE"
        },
        "run_if": "ALL_SUCCESS",
        "task_key": "success",
        "timeout_seconds": 0,
        "webhook_notifications": {}
      },
      {
        "depends_on": [
          {
            "task_key": "success"
          }
        ],
        "disable_auto_optimization": true,
        "email_notifications": {},
        "max_retries": 3,
        "min_retry_interval_millis": 300000,
        "notebook_task": {
          "notebook_path":
"/Workspace/Users/someone@example.com/directory",
```

```
          "source": "WORKSPACE"
        },
        "retry_on_timeout": false,
        "run_if": "ALL_SUCCESS",
        "task_key": "fail",
        "timeout_seconds": 0,
        "webhook_notifications": {}
      }
    ],
    "timeout_seconds": 0,
    "webhook_notifications": {}
  }
}
```

Example: Create a job using the CLI

The following example uses the Databricks CLI to create a job. This job contains a single job task that runs the specified notebook. This notebook has a dependency on a specific version of the `wheel` PyPI package. To run this task, the job temporarily creates a cluster that exports an environment variable named `PYSPARK_PYTHON`. After the job runs, the cluster is terminated.

1. Copy and paste the following JSON into a file. You can access the JSON format of any existing job by selecting the **View JSON** option from the job page UI.

   JSON

   ```
   {
     "name": "My hello notebook job",
     "tasks": [
       {
         "task_key": "my_hello_notebook_task",
         "notebook_task": {
           "notebook_path":
   "/Workspace/Users/someone@example.com/hello",
           "source": "WORKSPACE"
         }
       }
     ]
   }
   ```

2. Run the following command, replacing `<file-path>` with the path and name of the file that you just created.

Bash

```
databricks jobs create --json @<file-path>
```

Run a job using the CLI

There are three ways to run yoaur job when using the command line.

- **Scheduled**. If the job definition (in JSON) includes a schedule, like the following example, then the job will automatically run on the schedule.

  JSON

  ```
    "schedule": {
    "quartz_cron_expression": "46 0 9 * * ?",
    "timezone_id": "America/Los_Angeles",
    "pause_status": "UNPAUSED"
  },
  "max_concurrent_runs": 1,
  ```

- **Trigger with `run-now`**. The `databricks jobs run-now` CLI command triggers a run for a job that you have already created.

- **Trigger with `submit`**. The `databricks jobs submit` CLI command takes a job definition and triggers a run for the job.

  With `submit`, the job is not saved, and isn't visible in the UI. It runs once, and when it is done, it no longer exists as a job.

  Because they are not saved, submitted jobs can't be auto-optimized for serverless compute in case of failure. If your job fails, you may want to use classic compute to specify the compute needs for the job. Or use `jobs create` and `jobs run-now` to create and run the job.

Get started with the Databricks SDK

Databricks provides SDKs that allow you to automate operations using popular programming languages such as Python, Java, and Go. This section shows you how to get started using the Python SDK to create and manage jobs on Databricks.

You can use the Databricks SDK from your Databricks notebook or from your local development machine. If you are using your local development machine, ensure you first complete Get started with the Databricks SDK for Python.

> **NOTE**
>
> If you are developing from a Databricks notebook and are using a cluster that uses Databricks Runtime 12.2 LTS and below, you must install the Databricks SDK for Python first. See Install or upgrade the Databricks SDK for Python.

Example: Create a job using the Python SDK

The following example notebook code creates a job that runs an existing notebook. It retrieves the existing notebook's path and related job settings with prompts.

First, make sure that the correct version of the SDK has been installed:

Python

```
%pip install --upgrade databricks-sdk==0.74.0
%restart_python
```

Next, to create a job with a notebook task, run the following, answering the prompts:

Python

```
from databricks.sdk.service.jobs import JobSettings as Job
from databricks.sdk import WorkspaceClient

job_name            = input("Provide a short name for the job, for
example, my-job: ")
notebook_path       = input("Provide the workspace path of the notebook
to run, for example, /Users/someone@example.com/my-notebook: ")
task_key            = input("Provide a unique key to apply to the job's
tasks, for example, my-key: ")
```

```
test_sdk = Job.from_dict(
   {
       "name": job_name ,
       "tasks": [
           {
               "task_key": task_key,
               "notebook_task": {
                   "notebook_path": notebook_path,
                   "source": "WORKSPACE",
               },
           },
       ],
   }
)

w = WorkspaceClient()
j = w.jobs.create(**test_sdk.as_shallow_dict())

print(f"View the job at {w.config.host}/#job/{j.job_id}\n")
```

Run a job using the Python SDK

There are three ways to run a job when using the API.

- **Scheduled**. If the job definition (in JSON) includes a schedule, like the following example, then the job will automatically run on the schedule.

  JSON

  ```
    "schedule": {
    "quartz_cron_expression": "46 0 9 * * ?",
    "timezone_id": "America/Los_Angeles",
    "pause_status": "UNPAUSED"
  },
  "max_concurrent_runs": 1,
  ```

- **Trigger with `run-now`**. The `jobs.run_now` API triggers a run for a job that you have already created.

- **Trigger with `submit`**. The `jobs.runs.submit` API takes a job definition and triggers a run for the job.

With `submit`, the job is not saved, and isn't visible in the UI. It runs once, and when it is done, it no longer exists as a job.

Because they are not saved, submitted jobs can't be auto-optimized for serverless compute in case of failure. If your job fails, you may want to use classic compute to specify the compute needs for the job. Or use `jobs.create` and `jobs.run_now` to create and run the job.

Get started with the Databricks REST API

> **NOTE**
>
> Databricks recommends using the Databricks CLI and a Databricks SDK, unless you are using a programming language that does not have a corresponding Databricks SDK.

The following example makes a request to the Databricks REST API to retrieve details for a single job. It assumes the `DATABRICKS_HOST` and `DATABRICKS_TOKEN` environment variables have been set as described in Perform personal access token authentication.

Bash

```
$ curl --request GET "https://${DATABRICKS_HOST}/api/2.2/jobs/get" \
     --header "Authorization: Bearer ${DATABRICKS_TOKEN}" \
     --data '{ "job": "11223344" }'
```

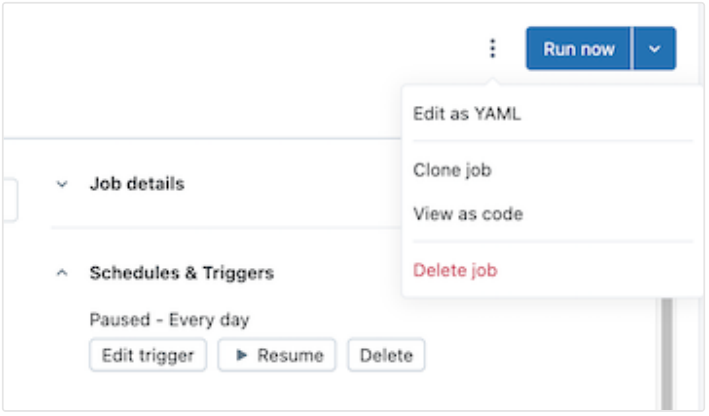
For information on using the Databricks REST API see the Databricks REST API reference documentation.

View jobs as code

From the Databricks workspace you can view the JSON, YAML, or Python representation of a job.

1. In your Databricks workspace's sidebar, click **Jobs & Pipelines** and select a job.

2. Click the kebab to the left of the **Run now** button, then click **View as code**:



3. Click **YAML**, **Python**, or **JSON** to view the job as code in that language.

   - For YAML, click **Copy**, then paste the code directly into Declarative Automation Bundles configuration `*.yaml` files to include the existing job in a bundle. You can also click **Edit** to modify the job's configuration in YAML instead of the UI.
   - For Python, choose **Databricks SDK** or **Declarative Automation Bundles**, then click **Copy**.
     - The Python code for the SDK can be used to create the job in notebooks or locally when using the Databricks Python SDK. See Create a job using the Python SDK.
     - The Python code for bundles can be used to include the job in a bundle using Python. See Bundle configuration in Python.
   - For JSON, click **Copy** and use the code to create, update, or get the job using the Databricks CLI, the Databricks SDKs, or the Databricks REST API.

Clean up

To delete any jobs you just created, run `databricks jobs delete <job-id>` from the Databricks CLI or delete the job directly from the Databricks workspace UI.

Next steps

- To learn more about the Databricks CLI, see What is the Databricks CLI? and Databricks CLI commands to learn about other command groups.

- To learn more about the Databricks SDK, see Databricks SDKs.
- To learn more about CI/CD using Databricks refer to Declarative Automation Bundles and Databricks Terraform provider for Databricks.
- For a comprehensive overview of all developer tooling, see Local development tools.